
Lab Manual

CSC 262

Introduction to Artificial Intelligence



Department of Computer Science

Islamabad Campus

CUI

Lab Contents:

The topics include: Basics of Python for AI , Different Searching techniques categorized into heuristical and blind , Analytical problems for testing AI approaches like N-Queens, Introduction to Genetic Algorithms , Introduction to Game Theory including min-max and alpha-beta pruning , and Game Modules Development.

Student Outcomes (SO)

S.#	Description
2	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
3	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations
4	Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations
5	Function effectively as an individual and as a member or leader in diverse teams and in multi-disciplinary settings.

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -5	Implement fundamental AI algorithms and AI techniques using appropriate software tools to solve intelligent system problems.	<i>Applying</i>	2-4
CLO-6	Develop an AI-based solution for a real-world problem by integrating appropriate AI techniques.	<i>Creating</i>	2-5

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab #	Main Topic	Page #
Lab 01	Python Basics for Artificial Intelligence	5
Lab 02	Understanding Data and Memory usage for AI	15
Lab 03	Uninformed Searching Algorithms: Depth First Search	35
Lab 04	Uninformed Searching Algorithm: Breadth First Search and Uniform Cost Search	42
Lab 05	Informed Search and Greedy Best-First Search	50
Lab 06	Informed Search Algorithms: A* Search	57
Lab 07	Local Search Algorithms: Hill Climbing , Simulated Annealing	69
Lab 8	Mid Term Exam	
Lab 9	*Introduction to Genetic Algorithms	
Lab 10	*Introduction to Game Theory: Min-Max and Alpha-Beta Pruning	
Lab 11	*Introduction to COLAB: Implementing a basic prediction system*	
Lab 12	*Linear Regression: Experimenting with different Datasets	
Lab 13	*Introduction to game engines using Python	
Lab 14	*Experimenting with graph database for knowledge representation	
Lab 15	*Creating Complex knowledge bases in Expert Systems*	
Lab 16	Final Term Exam	

*In progress

Lab 01: Python Basics for Artificial Intelligence	5
Lab 02: Understanding Data and Memory usage for AI.....	15
Lab 03 Uninformed Searching Algorithms: Depth First Search.....	35
Lab 04: Uninformed Searching Algorithm: Breadth First Search and Uniform Cost Search.....	49
Lab 05 Informed Search and Greedy Best-First Search	60
Lab 06 Informed Searching Algorithms: A* Search.....	67
Lab 07 Local Search Algorithms: Hill Climbing , Simulated Annealing	79
Lab 08 Midterm	90

Lab 01: Python Basics for Artificial Intelligence

Objective:

This lab is an introductory session on **Python**. It is a powerful object-oriented programming language, comparable to Perl, Ruby, Scheme and Java. This lab will enable the students to learn the core building blocks used in AI based approaches.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understanding IDE's
- Basic use of conditionals
- Basic use of loops.

Instructor Note:

As a pre-lab activity, read the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 30 minutes)

Programming is simply the act of entering instructions for the computer to perform. These instructions might crunch some numbers, modify text, look up information in files, or communicate with other computers over the Internet. All programs use basic instructions as building blocks.

Python refers to the Python programming language (with syntax rules for writing what is considered valid Python code) and the Python interpreter software that reads source code (written in the Python language) and performs its instructions. The name Python comes from the surreal British comedy group Monty Python, not from the snake. Python is a high-level programming language and is widely being used among the developers' community. Python was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code. Python is a programming language that lets developers work quickly and integrate systems more efficiently.

Before exploring Python's basics, data types and other features, let us explore why you should learn Python and how it has made its place in industry.

Applications :

1) GUI based desktop applications (Games, Science)

-
-
- 2) Web frameworks and applications
 - 3) Enterprise and Business applications
 - 4) Operating Systems
 - 5) Language Development
 - 6) Prototyping

Organizations using Python Language :

- 1) Google(Components of Google spider and Search Engine)
- 2) Yahoo(Maps)
- 3) YouTube
- 4) Mozilla
- 5) Dropbox
- 6) Microsoft

Python was developed by Guido van Rossum in the early 1990s and its latest version is 3.7.~, we can simply call it Python3. Python 3.0 and is interpreted language i.e it's not compiled and the interpreter will check the code line by line.

Solved Lab Activities:

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>2</i>	<i>2 Hours</i>	<i>Medium</i>	<i>CLO-5</i>

2) Solved Lab Activities (Allocated Time 2 Hours)

Activity 1: Display numbers on screen using Python IDLE.

Solution:

1. Run Python IDLE
2. Type in any number, say **24** and press **Enter**.
3. **24** should be printed on the screen

-
-
4. Now type **4.2**, press **Enter**.
 5. **4.2** should be displayed on screen as an output.
 6. Now type **print(234)**. Press **Enter**. **234** will be the output.
 7. Type **print(45.90)** and press **Enter**. The output will show **45.90** on screen.

Activity 2:

Display strings on screen.

Solution:

1. Python recognized strings through quotes (single, double). Anything inside quotes is a string for the Python interpreter.
2. Type **hello**, press **Enter**. An error message will be displayed as the Python interpreter does not understand this as a string.
3. Type **'hello'** and press **Enter**. Hello will be displayed.
4. Type **'Quote me on this!'** and press **Enter**. Same string will be displayed.
5. Type **"What's your name?"** and press **Enter**. **What's your name?** will be printed on the screen.
6. You can specify multi-line strings using triple quotes – **“ “ “** or **‘ ‘ ‘**. Type following text and press Enter

```
"""This is a multi-line string. This is the first line.  
This is the second line.  
"What's your name?," I asked.  
He said "Bond, James Bond." """
```

Activity 3:

Use Python as a calculator.

Solution:

1. The interpreter acts as a simple calculator: you can type an expression at it and it will write the value. Expression syntax is straightforward: the operators **+** (addition), **-** (subtraction), ***** (multiplication) and **/** (division) work just like in most other languages. Parentheses **()** can be used for grouping.
2. Type in **2 + 2** and press **Enter**. Python will output its result i.e. **4**.
3. Try following expressions in the shell.
 - a) **50 – 4**
 - b) **23.5 – 2.0**

-
-
- c) $23 - 18.5$
 - d) $5 * 6$
 - e) $2.5 * 10$
 - f) $2.5 * 2.5$
 - g) $28 / 4$ (note: division returns a floating point number)
 - h) $26 / 4$

Activity 4:

Get an integer answer from division operation. Also get remainder of a division operation in the output.

Solution:

1. Division (/) always gives a float answer. There are different ways to get a whole answer.
2. Use // operator:
 - a. Type **28 // 4** and press **Enter**. The answer is a whole number.
 - b. Type **26 // 4** and press **Enter**.
3. Use (int) cast operator: this operator changes the interchangeable types.
 - a. Type **print(int (26/4))** and press **Enter**. The answer is a whole number.
 - b. Type **print(int (28/6))** press **Enter**.
4. The modulus (or mod) % operator is used to get the remainder as an output (division / operator returns the quotient).
 - a. Type **28 % 4**. Press **Enter**. **0** will be the result.
 - b. Type **26 % 4**. Press **Enter**. **2** will be the result.

Activity 5:

Calculate 4^3 , 4^{10} , 4^{29} , 4^{150} , 4^{1000}

Solution:

1. The multiplication (*) operator can be used for calculating powers of a number. However, if the power is big, the task will be tedious. For calculating powers of a number, Python uses ** operator.
2. Type following and obtain the results of above expressions.
 - A. **4 ** 3**

-
-
- B. `4 ** 10`
 - C. `4 ** 29`
 - D. `4 ** 150`
 - E. `4 ** 1000`

Activity 6: Write following math expressions. Solve them by hand using operators' precedence.

Calculate their answers using Python. Match the results.

Solution:

The table below shows the operator precedence in Python (from Highest to Lowest).

Operator	Operation	Example
<code>**</code>	Exponent	<code>2 ** 3 = 8</code>
<code>%</code>	Modulus/remainder	<code>22 % 8 = 6</code>
<code>//</code>	Integer division/floored quotient	<code>22//8 = 2</code>
<code>/</code>	Division	<code>22/8 = 2.75</code>
<code>*</code>	Multiplication	<code>3*5 = 15</code>
<code>-</code>	Subtraction	<code>5 - 2 = 3</code>
<code>+</code>	Addition	<code>2 + 2 = 4</code>

Calculate following expressions:

- `2+3*6`
- `(2+3)*6`
- `48565878 * 578453`
- `2 + 2` (note the spaces after +)
- `(5 - 1) * ((7 + 1) / (3 - 1))`
- `5 + 7.42 + 5 + * 2`

Activity 7: Take input from the keyboard and use it in your program.

Solution: In Python and many other programming languages you can get user input. In Python the input() function will ask keyboard input from the user. The input function prompts text if a parameter is given. The function reads input from the keyboard, converts it to a string and removes the newline (Enter). Type and experiment with the script below.

```
name = input('What is your name? ')
print('Hello ' + name)
job = input('What is your job? ')
print('Your job is ' + job)
num = input('Give me a number? ')
print('You said: ' + str(num))
```

Activity 8:

Let us take an integer from the user as input and check whether the given value is even or not.

Solution:

- A. Create a new Python file from Python Shell and type the following code.
- B. Run the code by pressing F5.

```
n=input("Enter a number ")
if int(n)%2==0:
    print("The given number is an even number")
```

You will get the following output.

```
Enter a number 12
The given number is an even number
>>>
```

Activity 9:

Let us modify the code to take an integer from the user as input and check whether the given value is even or odd. If the given value is not even then it means that it will be odd. So here we need to use the if-else statement as demonstrated below.

Solution:

- A. Create a new Python file from Python Shell and type the following code.

B. Run the code by pressing F5.

```
n=input("Enter a number ")
if int(n)%2==0:
    print("The given number is an even number")
else:
    print("The given number is an odd number")
```

You will get the following output.

```
Enter a number 11
The given number is an odd number
>>>
```

Activity 10:

Calculate the sum of all the values between 0-10 using while loop.

Solution: Create a new Python file from Python Shell and type the following code.

```
summation = 0
i=1
while i<=10:
    summation=summation+i
    i=i+1

print("sum is ",summation)
```

You will get the following output.

```
sum is  55
>>>
```

Activity 11:

Accept 5 integer values from the user and display their sum. Draw flowchart before coding in python.

Solution:

```
summ = 0
i=0
while i<=4:
    s=input("enter a number")
    n=int(s)
    summ=summ+n
    i=i+1

print("sum is ",summ)
```

You will get the following output.

```
enter a number1
enter a number2
enter a number3
enter a number4
enter a number5
sum is  15
>>>
```

Activity 12:

Write a Python code to keep accepting integer values from user until 0 is entered. Display sum of the given values.

Solution:

```
sum=0
s=input("Enter an integer value...")
n=int(s)
while n!=0:
    sum=sum+n
    s=input("Enter an integer value...")
    n=int(s)
print("Sum of given values is ",sum)
```

You will get the following output.

```
Enter an integer value...10
Enter an integer value...521
Enter an integer value...5
Enter an integer value...22
Enter an integer value...0
Sum of given values is  558
>>>
```

Activity 13:

Write a Python code to accept an integer value from the user and check whether the given value is prime number or not.

Solution:

Create a new Python file from Python Shell and type the following code.

```
isPrime = True
i=2
n=int(input("enter a number"))
while i<n:
    remainder=n%i
    if remainder==0:
        isPrime=False
        break
    else:
        i=i+1

if isPrime:
    print("Number is Prime")
else:
    print("Number is not Prime")
```

You will get the following output(s).

```
enter a number17
Number is Prime
>>>

enter a number49
Number is not Prime
>>> |
```

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

Design and implement a **Simple Reflex Agent** in Python that controls an air conditioner based on the **current room temperature**.

The room temperature is the agent's **only percept**. The agent should use a simple **if-else** rule to decide whether to turn the air conditioner ON or OFF.

The agent follows this rule:

- If the room temperature is **greater than 28°C**, turn the **air conditioner ON**.
- Otherwise, turn the **air conditioner OFF**.

The program should accept the current room temperature as input and display the **percept (temperature)** and the **action selected by the agent**.

Lab Task 2:

Write a program in python that takes a number from the user and calculate the factorial of that number.

Lab Task 3:

The Fibonacci series is that when you add the previous two numbers the next number is formed.

You have to start from 0 and 1.

E.g. $0+1=1 \rightarrow 1+1=2 \rightarrow 1+2=3 \rightarrow 2+3=5 \rightarrow 3+5=8 \rightarrow 5+8=13$

So the series becomes

0 1 1 2 3 5 8 13 21 34 55

Steps: You have to take an input number that shows how many terms to be displayed. Then use loops for displaying the Fibonacci series up to that term e.g. input no is =6 the output should be 0 1 1 2 3 5.

Lab 02: Understanding Data and Memory usage for AI

Objective:

This lab will give you practical implementation of different types of sequences including lists, tuples, sets and dictionaries. We will use lists alongside loops in order to know about indexing individual items of these containers. This lab will also allow students to write their own functions.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Develop a basic understanding of data structures commonly used for AI
- Understanding Lists, sets, tuples and dictionaries
- Using loops with lists
- Developing customized functions for logical inference
- Basic understanding of Object Oriented Concepts like Class definitions etc.

Instructor Note:

As a pre-lab activity, the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 1 Hour)

In Python the concept of Lists is used which is just like arrays, declared in other languages which is an ordered collection of data. It is very flexible as the items in a list do not need to be of the same type.

The implementation of Python List is similar to Vectors in C++ or ArrayList in JAVA. The costly operation is inserting or deleting the element from the beginning of the List as all the elements are needed to be shifted. Insertion and deletion at the end of the list can also become costly in the case where the pre-allocated memory becomes full.

Python provides different types of data structures as sequences. In a sequence, there are more than one values and each value has its own index as can be seen in Figure 1 below:

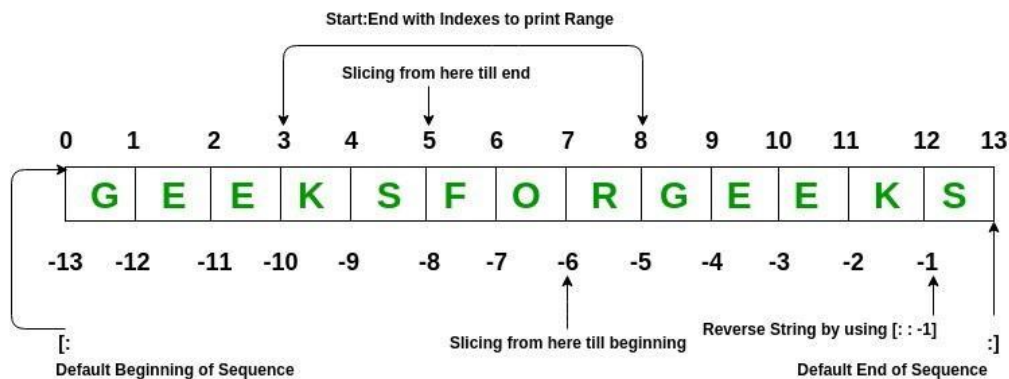


Figure 1: Assigning values in a List (Image Courtesy: <https://www.geeksforgeeks.org/python-data-structures/>)

The first value will have an index 0 in python, the second value will have index 1 and so on. These indices are used to access a particular value in the sequence.

Python offers different types of sequences. Lists are the most important type of sequence being used in Python. It is a collection of the same or different types of objects. These objects are separated by commas to distinguish from each other enclosed in square brackets. The following activities show how lists are used in Python.

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets. The data type "set", which is a collection type, has been part of Python since version 2.4. A set contains an unordered collection of unique and immutable objects.

The set data type is, as the name implies, a Python implementation of the sets as they are known from mathematics. This explains why sets unlike lists or tuples can't have multiple occurrences of the same element.

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values.

Solved Lab Activities:

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>2</i>	<i>2 Hours</i>	<i>Medium</i>	<i>CLO-5</i>

2) Solved Lab Activities (Allocated Time 2 Hours)

Activity 1:

For developing understanding, use loops to accept 5 values from the user and store them in a list. Display all the values (objects) of the list.

```
myList=[]
for i in range(5):
    val=input("Enter a value:")
    myList.append(val)
print("The given list is")
print(myList)
```

The output should be similar to the following:

```
Enter a value:1
Enter a value:Mango
Enter a value:Fruit
Enter a value:Pakistan
Enter a value:1947
The given list is
['1', 'Mango', 'Fruit', 'Pakistan', '1947']
>>>
```

Activity 2:

Accept 5 integer values from the user. Store these values in a list and display the list in ascending order.

```
myList=[]
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList.append(n)

myList.sort()
print(myList)
```

Notice the use of sort

The output in this case would be:

```
Enter a value:1
Enter a value:5
Enter a value:2
Enter a value:4
Enter a value:3
[1, 2, 3, 4, 5]
>>>
```

Activity 3:

Accept two lists from the user and display their join.

```
myList1=[]
print("Enter objects of first list...")
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList1.append(n)

myList2=[]
print("Enter objects of second list...")
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList2.append(n)

list3=myList1+myList2;
print(list3)
```

The output should be:

```
Enter objects of first list...
Enter a value:1
Enter a value:2
Enter a value:3
Enter a value:4
Enter a value:5
Enter objects of second list...
Enter a value:6
Enter a value:7
Enter a value:8
Enter a value:9
Enter a value:0
[1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
>>>
```

Activity 4:

Nesting a list inside of a list and accessing elements.

```
# Creating a List with
# the use of multiple values
List = ["Geeks", "For", "Geeks"]
print("\nList containing multiple values: ")
print(List)
# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List2 = [['Geeks', 'For'], ['Geeks']]
print("\nMulti-Dimensional List: ")
print(List2)
# accessing an element from the # list using index number
print("Accessing element from the list")
print(List[0])
print(List[2])
# accessing an element using # negative indexing
print("Accessing element using negative indexing")

# print the last element of list
print(List[-1])

# print the third last element of list
print(List[-3])
```

Output should be:

List containing multiple values:

['Geeks', 'For', 'Geeks']

Multi-Dimensional List:

[['Geeks', 'For'], ['Geeks']]

Accessing element from the list

Geeks

Geeks

Accessing element using negative indexing

Geeks

Geek

s

Activity 5:

Creating a dictionary in Python, basically a Python dictionary is like hash tables in any other language. It is an unordered collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element, Dictionary holds the key:value pair. Keyvalue is provided in the dictionary to make it more optimized.

Indexing of the Python Dictionary is done with the help of keys. These are of any hashable type i.e. an object which can never change like strings, numbers, tuples, etc. We can create a dictionary by using curly braces ({}) or dictionary comprehension.

```
# Creating a Dictionary
Dict = {'Name': 'Geeks', 1: [1, 2, 3, 4]}
print("Creating Dictionary: ")
print(Dict)
# accessing a element using key
print("Accessing an element using key:")
print(Dict['Name'])
# accessing a element using get() method
print("Accessing an element using get:")
print(Dict.get(1))
# creation using Dictionary comprehension
myDict = {x: x**2 for x in [1,2,3,4,5]}
print(myDict)
```

The output should be:

Creating Dictionary:

```
{'Name': 'Geeks', 1: [1, 2, 3, 4]}
```

#Accessing an element using key:

```
Geeks
```

#Accessing an element using get:

```
[1, 2, 3, 4]
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

The dictionary can also be used with some added in-built features like `OrderedDict`. An `OrderedDict` is also a sub-class of dictionary but unlike a dictionary, it remembers the order in which the keys were inserted.

```
from collections import OrderedDict

print("Before deleting:\n")
od = OrderedDict()
od['a'] = 1
od['b'] = 2
od['c'] = 3
od['d'] = 4
for key, value in od.items():
    print(key, value)

print("\nAfter deleting:\n")
od.pop('c')
for key, value in od.items():
    print(key, value)

print("\nAfter re-inserting:\n")
od['c'] = 3
for key, value in od.items():
    print(key, value)
```

In this case the output would be:

Before deleting:

a 1
b 2
c 3
d 4

After deleting:

a 1
b 2
d 4

After re-inserting:

a 1
b 2
d 4

c 3

Similarly for combining many dictionaries we can use a Chainmap. A ChainMap encapsulates many dictionaries into a single unit and returns a list of dictionaries. When a key is needed to be found then all the dictionaries are searched one by one until the key is found.

```
from collections import ChainMap

d1 = {'a': 1, 'b': 2}
d2 = {'c': 3, 'd': 4}
d3 = {'e': 5, 'f': 6}

# Defining the chainmap
c = ChainMap(d1, d2, d3)
print(c)
print(c['a'])
print(c['f'])
```

In this case the output would be:

```
ChainMap({'a': 1, 'b': 2}, {'c': 3, 'd': 4}, {'e': 5, 'f': 6})
```

```
1
```

```
6
```

Note: the missing key returned an error as if it was not able to find it in dictionaries.

Activity 6:

The Python Tuple is a collection of Python objects much like a list but Tuples are immutable in nature i.e. the elements in the tuple cannot be added or removed once created. Just like a List, a Tuple can also contain elements of various types.

In Python, tuples are created by placing a sequence of values separated by ‘comma’ with or without the use of parentheses for grouping of the data sequence.

```
# Creating a Tuple with the use of Strings
Tuple = ('Geeks', 'For')
print("\nTuple with the use of String: ")
print(Tuple)

# Creating a Tuple with the use of list
list1 = [1, 2, 4, 5, 6]
print("\nTuple using List: ")
Tuple = tuple(list1)
```

```
# Accessing element using indexing
print("First element of tuple")
print(Tuple[0])
# Accessing element from last negative indexing
print("\nLast element of tuple")
print(Tuple[-1])

print("\nThird last element of tuple")
print(Tuple[-3])
```

The output should be:

Tuple with the use of String:

('Geeks', 'For')

Tuple using List:

First element of tuple

1

Last element of tuple

6

Third last element of tuple

4

The Tuple also has an enhanced version called NamedTuple, which returns a tuple object with names for each position which the ordinary tuples lack.

For example, consider a tuple name student where the first element represents name, second represents age and the third element represents the DOB. Suppose for calling name instead of remembering the index position you can actually call the element by using the name argument, then it will be really easy for accessing tuple element. This functionality is provided by the namedtuple.

```
from collections import namedtuple
# Declaring namedtuple()
Student = namedtuple('Student', ['name', 'age', 'DOB'])
# Adding values
S = Student('Ali', '18', '2542004')
# Access using index
```

```
print ("The Student age using index is : ",end = "")
print (S[1])
# Access using name
print ("The Student name using keyname is : ",end = "")
print (S.name)
```

The output would be:

The Student age using index is : 18

The Student name using keyname is : Ali

Activity 7:

In Python, a **set** is a built-in data type used to store a collection of unique, unordered, and unindexed items. For example:

```
my_set = {10, 20, 30, 40}
```

```
print(my_set)
```

They are highly efficient for removing duplicates from data sequences and performing mathematical set operations like unions and intersections. Sets are basically used to include membership testing and eliminating duplicate entries. A set can store multiple data types (integers, strings, booleans). However, its elements must be **immutable/hashable**; you can put a tuple inside a set, but you cannot put a mutable list or another set inside it.

```
# Creating a set with elements
```

```
fruits = {"apple", "banana", "cherry"}
```

```
# Duplicates are automatically removed
```

```
numbers = {1, 2, 2, 3, 4, 4}
```

```
print(numbers) # Output: {1, 2, 3, 4}
```

```
empty_set = set()# Creating an empty set
```

```
# Creating a Set with # a mixed type of values
# (Having numbers and strings)
Set = set([1, 2, 'Geeks', 4, 'For', 6, 'Geeks'])
print("\nSet with the use of Mixed Values")
print(Set)
# Accessing element using for loop
print("\nElements of set: ")
for i in Set:
    print(i, end = " ")
print()
# Checking the element using in keyword
print("Geeks" in Set)
```

The output should be:

Set with the use of Mixed Values

{1, 2, 'Geeks', 4, 6, 'For'}

Elements of set:

1 2 Geeks 4 6 For

True

Try also experimenting with frozen sets.

Activity 8:

The Python Strings are arrays of bytes representing Unicode characters. In simpler terms, a string is an immutable array of characters. Python does not have a character data type, a single character is simply a string with a length of 1. As strings are immutable, modifying a string will result in creating a new copy.

```
String = "Welcome to GeeksForGeeks"
print("Creating String: ")
print(String)

# Printing First character
print("\nFirst character of String is: ")
print(String[0])

# Printing Last character
```

```
print("\nLast character of String is: ", String[-1])
```

The output should be:

Creating String:

Welcome to GeeksForGeeks

First character of String is:

W

Last character of String is: s

Activity 9:

It is especially important to understand memory allocations in Python. As python is used in ML and AI where large datasets are used. Memory leaks,i.e, the program is out of memory after running for several hours, is a common problem. To manage these memory leaks memory monitoring is essential. Monitoring memory is also called profiling. As a developer, it's a necessity that we profile our program and use less memory allocation as much as possible.

For this purpose one method is Tracemalloc which is a library module that traces every memory block in python. The tracing starts by using the start() during runtime. This library module can also give information about the total size, number, and average size of allocated memory blocks.

In the first instance we can use the tracemalloc library for a reference problem.

```
# importing the module
import tracemalloc
# code or function for which memory # has to be monitored
def app():
    lt = []
    for i in range(0, 100000):
        lt.append(i)
```

```
# starting the monitoring
tracemalloc.start()
# function call
app()
# displaying the memory
print(tracemalloc.get_traced_memory())
# stopping the library
tracemalloc.stop()
```

The output is given in form of (current, peak), i.e., current memory is the memory the code is currently using and peak memory is the maximum space the program used while executing.

(3200, 3992224)

3) Graded Lab Tasks (Allotted Time 40 minutes)

Lab Task 1:

Design and implement a Simple Reflex Agent in Python that makes decisions based only on its current percepts using a predefined set of if-else rules.

Consider an AI-based smart room agent responsible for maintaining a comfortable room environment. The agent receives percepts from sensors, such as the current temperature and light level, and takes an appropriate action based on these percepts.

The agent should follow rules such as:

- If the temperature is above 30°C, the agent turns ON the fan.
- If the temperature is 30°C or below, the agent turns OFF the fan.
- If the light level is low, the agent turns ON the light.
- If the light level is sufficient, the agent turns OFF the light.

The program should represent the agent's percepts, internal state, and actions using appropriate Python variables or data structures. It should accept sensor percepts as input, apply the predefined if-else rules, and display the agent's current percepts, state, and selected actions.

Object Oriented Programming in Python:

Python is an object-oriented language, allowing you to structure your code using classes and objects for better organization and reusability.

Advantages of OOP

- Provides a clear structure to programs
- Makes code easier to maintain, reuse, and debug
- Helps keep your code DRY (**Don't Repeat Yourself**)
- Allows you to build reusable applications with less code

What are Classes and Objects?

Classes and objects are the two core concepts in object-oriented programming.

A class defines what an object should look like, and an object is created based on that class. For example:

Class	Objects
Fruit	Apple, Banana, Mango
Car	Volvo, Audi, Toyota

When you create an object from a class, it inherits all the variables and functions defined inside that class.

Python Classes/Objects:

Python is an object oriented programming language. Almost everything in Python is an object, with its properties and methods. A Class is like an object constructor, or a "blueprint" for creating objects.

Create a class named MyClass, with a property named x:

```
class MyClass:
    x = 5

#Create an object named p1, and print the value of x:
```

```
p1 = MyClass()  
print(p1.x)
```

Output:

5

Create three objects from the MyClass class:

```
class MyClass:  
    x = 5  
  
p1 = MyClass()  
p2 = MyClass()  
p3 = MyClass()  
  
print(p1.x)  
print(p2.x)  
print(p3.x)
```

Output:

5

5

5

The __init__() Method:

All classes have a built-in method called `__init__()`, which is always executed when the class is being initiated.

The `__init__()` method is used to assign values to object properties, or to perform operations that are necessary when the object is being created.

Example:

Create a class named Person, use the `__init__()` method to assign values for name and age:

```
class Person:

    def __init__(self, name, age):

        self.name = name

        self.age = age

p1 = Person("Emili", 36)

print(p1.name)

print(p1.age)
```

Output:

Emili

36

Note: The `__init__()` method is called automatically every time the class is being used to create a new object. class definitions cannot be empty, but if you for some reason have a class definition with no content, put in the pass statement to avoid getting an error .Without the `__init__()` method, you would need to set properties manually for each object:

Create a class without `__init__()`:

```
class Person:

    pass

p1 = Person()

p1.name = "Tooba"

p1.age = 25

print(p1.name)

print(p1.age)
```

Output: Tooba

25

Using `__init__()` makes it easier to create objects with initial values. With `__init__()`, you can set initial values when creating the object:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Linus", 28)

print(p1.name)
print(p1.age)
```

Output:

Linus

28

Set a default value for the age parameter: You can also set default values for parameters in the `__init__()` method.

```
class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age

p1 = Person("Emili")
p2 = Person("Ali")

print(p1.name, p1.age)
print(p2.name, p2.age)
```

Output:

Emili 18

Ali 18

Create a Person class with multiple parameters:

```
class Person:

    def __init__(self, name, age, city, country):

        self.name = name

        self.age = age

        self.city = city

        self.country = country

p1 = Person("Linus", 30, "Oslo", "Norway")

print(p1.name)

print(p1.age)

print(p1.city)

print(p1.country)
```

Output:

Linus

30

Oslo

Norway

3) Graded Lab Tasks (Allotted Time 1 Hour)

Lab Task 2:

Let us put our learning to test now, and solve the following given problems:

- i. Write a Python program to create an instance of a specified class and display the namespace of the said instance.
- ii. Write a Python function `student_data ()` which will print the id of a student (`student_id`). If the user passes an argument `student_name` or `student_class` the function will print the student name and class
- iii. Write a simple Python class named `Student` and display its type. Also, display the `__dict__` attribute keys and the value of the `__module__` attribute of the `Student` class.

-
-
- iv. Write a Python program to create two empty classes, Student and Marks. Now create some instances and check whether they are instances of the said classes or not. Also, check whether the said classes are subclasses of the built-in object class or not
 - v. Write a Python class named Student with two attributes: student_name, marks. Modify the attribute values of the said class and print the original and modified values of the said attributes.
 - vi. Write a Python class named Student with two attributes student_id, student_name. Add a new attribute student_class and display the entire attribute and their values of the said class. Now remove the student_name attribute and display the entire attribute with values.
 - vii. Write a Python class named Student with two attributes student_id, student_name. Add a new attribute student_class. Create a function to display the entire attribute and their values in Student class.
 - viii. Write a Python class named Student with two instances student1, student2 and assign given values to the said instances attributes. Print all the attributes of student1, student2 instances with their values in the given format.
 - ix. Run Memory Allocation tests on the created Python class.
 - x. Try reducing the memory allocation using different techniques.

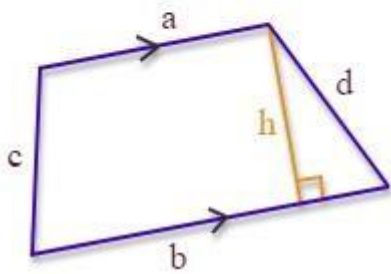
3) Graded Lab Tasks (Allotted Time 1 Hour)

Lab Task 3:

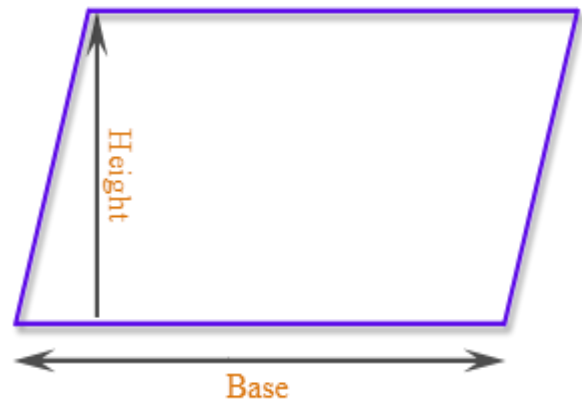
- i. Make two classes for trapezoid and parallelogram and write a code to calculate their areas.
- ii. Also write a comparison class to see which shape has the largest area, store results in a dictionary, iterate for different dimensions and use dictionary functions to search the shapes with largest areas.

Area of a Trapezoid

$$A = \frac{1}{2} (a + b) h$$



Parallelogram



Area of parallelogram = **Base** x **Height**

Lab 03 Uninformed Searching Algorithms: Depth First Search

Objective:

This lab will introduce students to search problems. We will first start by representing problems in terms of state space graph. Given a state space graph, starting state and a goal state, students will then perform a basic Depth First Search solution within that graph. The output will be a set of actions or a path that will begin from initial state/node and end in the goal node.

Useful Concepts (Allocated Time 15 Minutes)

. There are several types of blind or uninformed searches that can be used to solve a certain problem. Commonly used search techniques are:

- Breadth First Search
- Depth First Search
- Uniform Cost Search

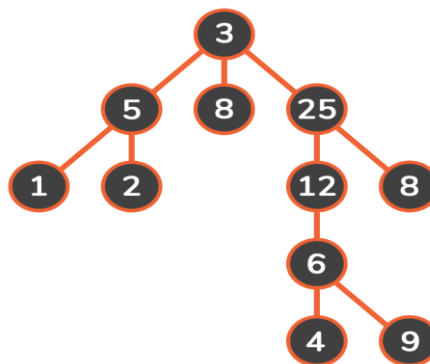


Figure: DFS Path [3-5-1-2-8-25-12-6-4-9-8]

Depth-First Search (DFS) is an algorithm used to traverse a graph or tree or search for a target node. It prioritizes depth by exploring one branch as far as possible. When no further node can be explored, it backtracks to the most recent node with an unexplored branch. This process continues until the goal is found or all reachable nodes have been explored.

DFS can be implemented recursively or iteratively. In a tree, the search normally begins at the root, whereas in a general graph, any node can be selected as the start node.

DFS is also used as a building block in several graph algorithms, including path finding, cycle detection, topological sorting, and finding connected components. Its simple traversal mechanism and recursive implementation make it useful for many graph-based problems.

Python libraries can also support the implementation and visualization of search problems. **NetworkX** is used to create and manipulate graphs, while **Matplotlib** can visualize graphs and search paths. **Pygame** can provide interactive visualization of grid-based search problems, including explored states, agent movement, and solution paths.

2) Solved Lab Activities (Allocated Time 2 Hour)

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
2	2 Hours	Medium	CLO-5

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Visualizing graphs using *NetworkX* and *matplotlib* library
- Implement depth first search technique
- Implement different path problems using depth first technique

Activity 1: Creating and Visualizing a Graph Using NetworkX

NetworkX is a Python library used for creating and working with graphs and networks. Matplotlib can be used with NetworkX to visualize these graphs.

Consider the following graph:

```
import networkx as nx
import matplotlib.pyplot as plt

# Create an undirected graph
G = nx.Graph()

# Add edges
G.add_edges_from([
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'D'),
    ('B', 'E'),
    ('C', 'F'),
    ('D', 'G'),
    ('E', 'G'),
    ('F', 'G')
])
```

```

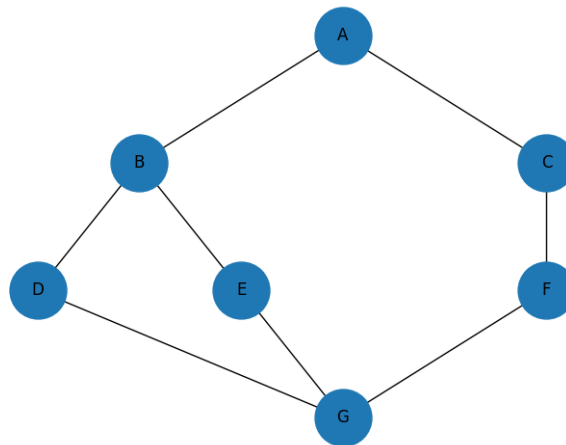
])

# Define positions of nodes
pos = {
    'A': (0, 3),
    'B': (-1, 2),
    'C': (1, 2),
    'D': (-1.5, 1),
    'E': (-0.5, 1),
    'F': (1, 1),
    'G': (0, 0)
}

# Draw the graph
nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=1800,
    font_size=12
)

plt.show()

```



Out put

Observe how `nx.Graph()` creates the graph, `add_edges_from()` defines the connections between nodes, and `nx.draw()` visualizes the resulting graph.

Activity 2: Adding Heuristic Values to Graph Nodes

In informed search, a heuristic function $h(n)$ estimates the cost or distance from a node n to the goal. (you will see in lab 5 onwards). Extend the graph from Activity 1 by assigning a heuristic value to each node.

```
heuristic = {
    'A': 7,
    'B': 5,
    'C': 3,
    'D': 4,
    'E': 2,
    'F': 1,
    'G': 0
}

# Store heuristic values as node attributes
nx.set_node_attributes(G, heuristic, 'heuristic')

print(G.nodes(data=True))
```

The heuristic values are now stored as attributes of the corresponding nodes. Since **G is the goal node**, its heuristic value is $h(G) = 0$.

Activity 3: Adding and Displaying Edge Costs

In many search problems, each edge has an associated cost, representing quantities such as distance, travel time, or resource usage. Extend the graph from Activity 1 by assigning costs to its edges.

```
import networkx as nx
import matplotlib.pyplot as plt

# Create a graph with edge costs
G = nx.Graph()
G.add_weighted_edges_from([
    ('A', 'B', 2),
    ('A', 'C', 4),
    ('B', 'D', 3),
    ('B', 'E', 5),
```

```

    ('C', 'F', 2),
    ('D', 'G', 4),
    ('E', 'G', 2),
    ('F', 'G', 3)
])

```

```

# Define positions of nodes

```

```

pos = {
    'A': (0, 3),
    'B': (-1, 2),
    'C': (1, 2),
    'D': (-1.5, 1),
    'E': (-0.5, 1),
    'F': (1, 1),
    'G': (0, 0)
}

```

```

# Draw nodes and edges

```

```

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=1800,
    font_size=12
)

```

```

# Get and display edge costs

```

```

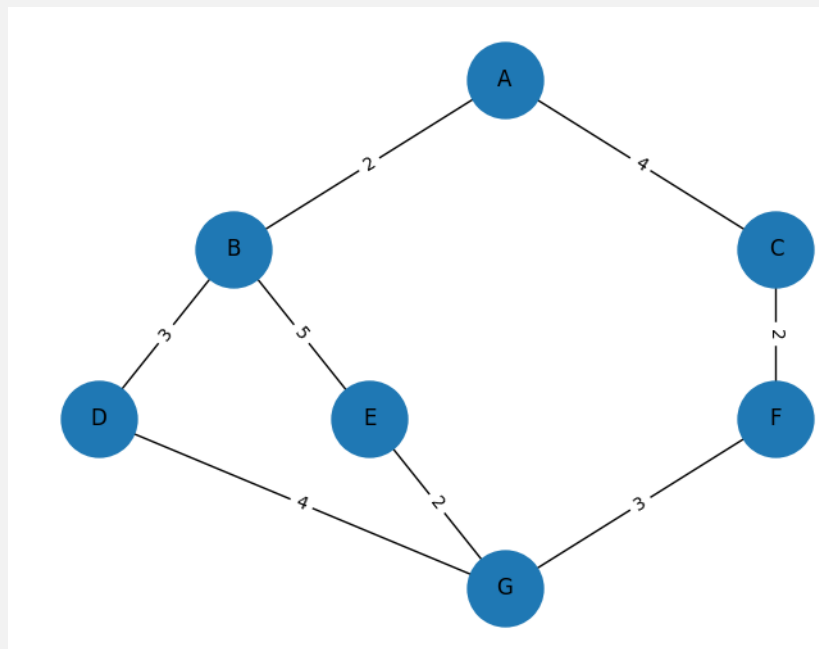
cost_labels = nx.get_edge_attributes(G, 'weight')

```

```

nx.draw_networkx_edge_labels(
    G,
    pos,

```



```
    edge_labels=cost_labels,  
    font_size=10  
)  
plt.show()
```

Here, each tuple such as: ('A', 'B', 2) represents an edge from **A to B with a cost of 2**. The resulting graph displays these costs alongside the corresponding edges.

Activity 04: Visualizing a Grid-Based Search Path Using Pygame

Objective:

To understand how Pygame can be used to represent a grid-based search problem by displaying the start state, goal state, and a solution path.

The following code is built in **four parts**. Each new part should be added below the previous one unless stated otherwise.

Part 1: Initialize Pygame and Define the Grid

Start a new Python file with:

```
import pygame  
  
pygame.init()  
  
ROWS = 4  
COLS = 4  
CELL_SIZE = 100  
  
WIDTH = COLS * CELL_SIZE  
HEIGHT = ROWS * CELL_SIZE  
  
screen = pygame.display.set_mode((WIDTH, HEIGHT))  
pygame.display.set_caption("Grid Search Visualization")  
  
clock = pygame.time.Clock()  
  
WHITE = (255, 255, 255)  
BLACK = (0, 0, 0)  
GREEN = (0, 200, 0)  
RED = (220, 0, 0)  
LIGHT_BLUE = (100, 180, 255)
```

This part initializes Pygame, creates a 4×4 window, and defines the colors that will be used in the visualization.

Part 2: Define the Start, Goal, and Path

Add the following code directly below Part 1:

```
start = (0, 0)
```

```
goal = (3, 3)
```

```
path = [
```

```
    (0, 0),
```

```
    (1, 0),
```

```
    (2, 0),
```

```
    (2, 1),
```

```
    (2, 2),
```

```
    (3, 2),
```

```
    (3, 3)
```

```
]
```

Each state is represented as (row, column). Here, path represents an example solution path from the start state to the goal state.

Part 3: Draw the Grid and Its States

Add this function below Part 2:

```
def draw_grid():
```

```
    screen.fill(WHITE)
```

```
    for row in range(ROWS):
```

```
        for col in range(COLS):
```

```
            cell = pygame.Rect(
                col * CELL_SIZE,
                row * CELL_SIZE,
                CELL_SIZE,
                CELL_SIZE
            )
```

```
            if (row, col) == start:
```

```
                color = GREEN
```

```
            elif (row, col) == goal:
```

```
                color = RED
```

```
            elif (row, col) in path:
```

```
                color = LIGHT_BLUE
```

```
            else:
```

```
                color = WHITE
```

```
pygame.draw.rect(screen, color, cell)
pygame.draw.rect(screen, BLACK, cell, 2)
```

The nested loops visit every grid cell. The start is shown in green, the goal in red, and the intermediate path cells in light blue.

Part 4: Keep the Visualization Running

Add the following code at the end of the program, below the `draw_grid()` function:

```
running = True

while running:

    # Check whether the user has closed the window
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    draw_grid()
    pygame.display.update()

    clock.tick(30)

# Pygame closes only after the user closes the window
pygame.quit()
```

The while running loop keeps the Pygame window active **indefinitely**. The program exits only when the user clicks the window's **Close (X)** button.

The expected visualization is:



Activity 05:

The Depth-First Search is a recursive algorithm that uses the concept of backtracking. It involves thorough searches of all the nodes by going ahead if potential, else by backtracking. Here, the word backtrack means once you are moving forward and there are not any more nodes along the present path, you progress backward on an equivalent path to seek out nodes to traverse. All the nodes are progressing to be visited on the current path until all the unvisited nodes are traversed after which subsequent paths are going to be selected.

Before learning the python code for Depth-First and its output, let us go through the algorithm it follows for the same. The recursive method of the Depth-First Search algorithm is implemented using stack.

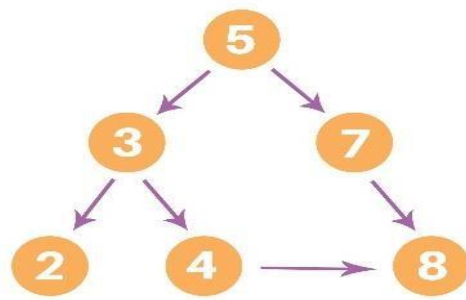


Figure: Graph for DFS implementation

The DSF traversal algorithm follows as:

Start node→Explore deeply→Visit all reachable nodes

We will start by putting any one of the graph's vertex on top of the stack.

After that take the top item of the stack and add it to the visited list of the vertex.

Next, create a list of that adjacent node of the vertex. Add the ones which aren't in the visited list of vertexes to the top of the stack.

Lastly, keep repeating steps 2 and 3 until the stack is empty.

```
# Using a Python dictionary to act as an adjacency list
graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}
```

```

# Set to keep track of visited nodes of graph
visited = set()
# Function for DFS
def dfs(visited, graph, node):
    if node not in visited:
        print(node)
        visited.add(node)

        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, '5')

```

The output in this case would be:

Following is the Depth-First Search

5 3 2 4 8 7

DFS with goal node:

In the previous example, DFS continues until all reachable nodes have been visited. In a search problem, however, we are interested in finding a particular goal node. DFS is modified to terminate when the specified goal node is reached.

Start node→Explore deeply→Goal test→Stop when goal is found or no more node left

```

graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = set()

# DFS with goal test
def dfs(visited, graph, node, goal):

```

```

if node not in visited:
    print(node)
    visited.add(node)

    # Goal test
    if node == goal:
        print("Goal found!")
        return True

    for neighbour in graph[node]:
        if dfs(visited, graph, neighbour, goal):
            return True

    return False
# Driver Code
print("DFS Search:")
dfs(visited, graph, '5', '4')

```

Output: DFS Search: 5 3 2 4 Goal found!

Activity 6:

The Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree. The only difference is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a Boolean visited array.

For example, in the graph given in the Figure below, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Depth First Traversal of the following graph is 2, 0, 1, 3.

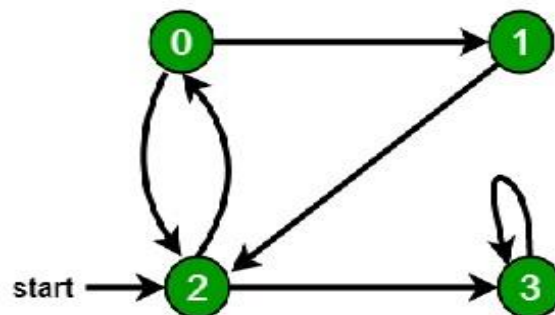


Figure: DFS Graph for implementation

```

# Python program for goal-based DFS

```

```
# Graph represented using an adjacency list
graph = {
    0: [1, 2],
    1: [2],
    2: [0, 3],
    3: [3]
}

# Set to keep track of visited nodes
visited = set()

# Function for DFS
def dfs(graph, node, goal, visited):

    # Check if node has already been visited
    if node not in visited:

        # Mark current node as visited
        visited.add(node)
        print(node, end=' ')

        # Check whether the goal node is reached
        if node == goal:
            print("\nGoal found!")
            return True

        # Visit all adjacent nodes
        for neighbour in graph[node]:
            if dfs(graph, neighbour, goal, visited):
                return True

    return False

# Driver Code
start_node = 0
goal_node = 3

print("Following is Goal-Based Depth First Search:")
dfs(graph, start_node, goal_node, visited)
```

Output: Following is Goal-Based Depth First Search:

0 1 2 3

Goal found!

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

To move onto more complex implementations, solve the route-finding search problem on the Romania road map. given in Figure below using DFS from Arad to Bucharest:

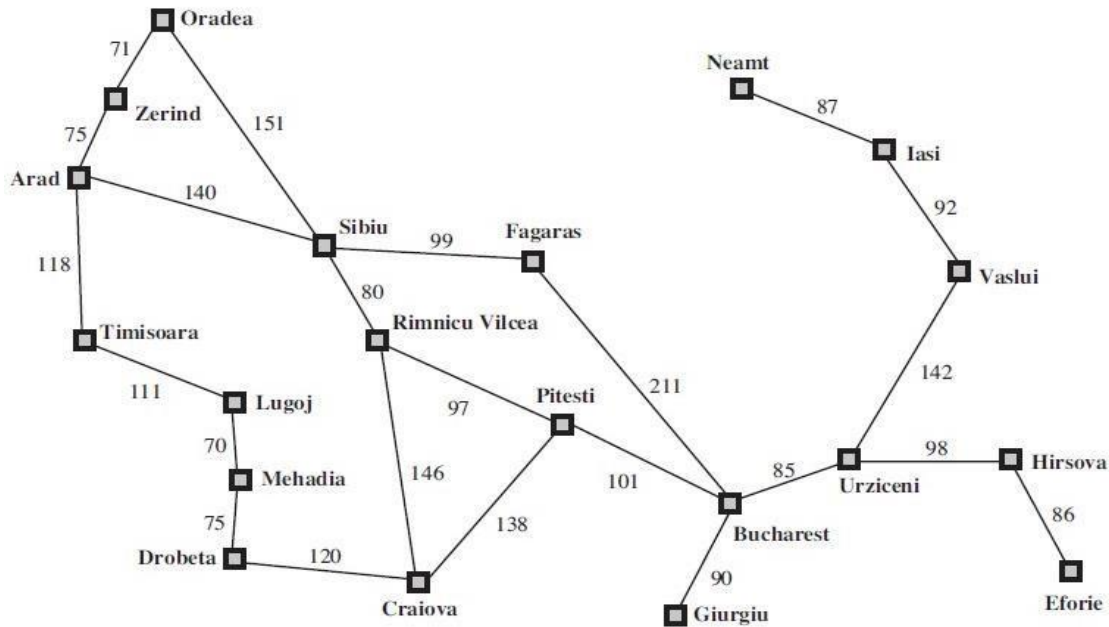


Figure: Graph for Route Finding in Romania map

- Draw a corresponding graph from the above Figure from Arad to Bucharest containing all possible nodes. (Arad as Start Node and Bucharest as Goal Node).
- Apply depth first search to reach the goal state, your output should have the path and the total distance to reach the path.
- Save your code and results as DFS_REG#.py as it would be used in later exercises.

Lab Task 2:

Generate a list of possible words from a character matrix. Given an $M \times N$ boggle board, find a list of all possible words that can be formed by a sequence of adjacent characters on the board.

We are allowed to search a word in all eight possible directions, i.e., North, West, South, East, NorthEast, North-West, South-East, South-West, but a word should not have multiple instances of the same cell.

Consider the following the traditional 4×4 boggle board. If the input dictionary is [START, NOTE, SAND, STONED], the valid words are [NOTE, SAND, STONED].

M	S	E	F
R	A	T	D
L	O	N	E
K	A	F	B

F 1 - 4x4

Expected out put:

M	S	E	F
R	A	T	D
L	O	N	E
K	A	F	B

Found Word: SAND

M	S	E	F
R	A	T	D
L	O	N	E
K	A	F	B

Found Word: STONED

M	S	E	F
R	A	T	D
L	O	N	E
K	A	F	B

Found Word: NOTE

Lab Task 3:

Create a graph of the Figure for the map given above and then set initial and goal states such that the number of nodes visited for BFS is smaller than that in DFS. Now modify the initial and goal state such that the number of nodes visited for BFS is larger than that in DFS (you can do this after next lab)

Lab 04: Uninformed Searching Algorithm: Breadth First Search and Uniform Cost Search

Objective:

This lab will introduce students to search problems. We will first start by representing problems in terms of state space graphs. Given a state space graph, starting state and a goal state, students will then perform a basic Breadth First Search solution within that graph. The output will be a set of actions or a path that will begin from the initial state/node and end in the goal node.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Develop a basic understanding of uninformed search techniques
- Understand breadth first search & uniform cost search.

Instructor Note:

As a pre-lab activity, read the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 40Minutes)

BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layerwise thus exploring the neighbour nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbour nodes. As the name BFS suggests, you are required to traverse the graph breadthwise as follows:

1. First move horizontally and visit all the nodes of the current layer
2. Move to the next layer

Consider the following diagram.

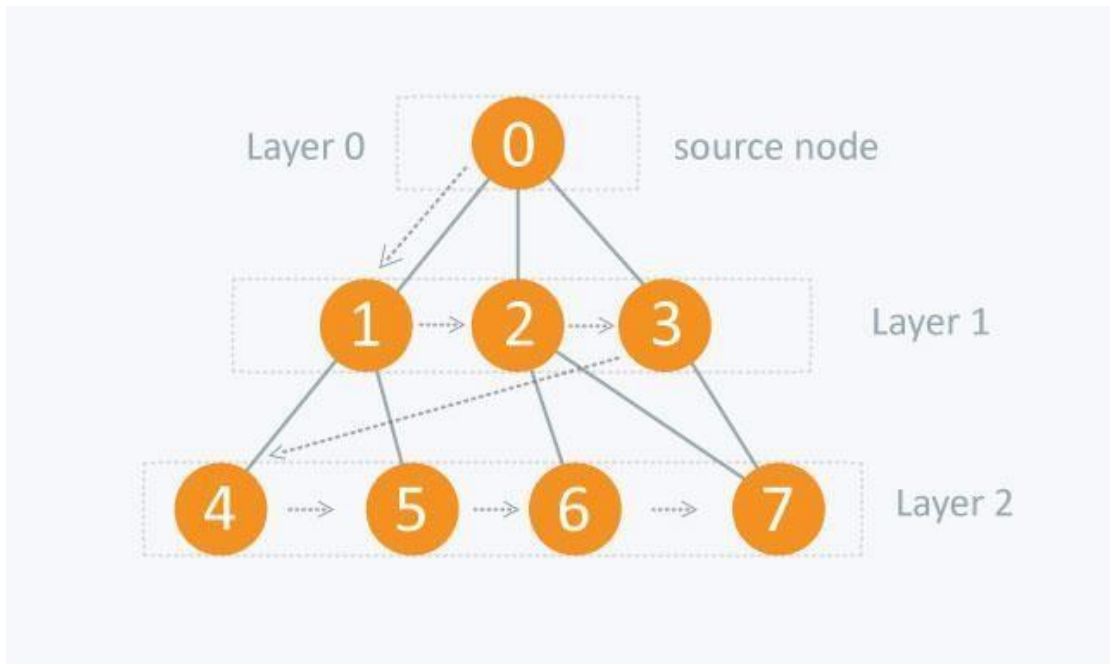


Figure - Graph

The distance between the nodes in layer 1 is comparatively lesser than the distance between the nodes in layer 2. Therefore, in BFS, you must traverse all the nodes in layer 1 before you move to the nodes in layer 2.

Traversing child nodes

A graph can contain cycles, which may bring you to the same node again while traversing the graph. To avoid processing the same node again, use a boolean array which marks the node after it is processed. While visiting the nodes in the layer of a graph, store them in a manner such that you can traverse the corresponding child nodes in a similar order.

In the earlier diagram, start traversing from 0 and visit its child nodes 1, 2, and 3. Store them in the order in which they are visited. This will allow you to visit the child nodes of 1 first (i.e. 4 and 5), then of 2 (i.e. 6 and 7), and then of 3 (i.e. 7) etc.

To make this process easy, use a queue to store the node and mark it as 'visited' until all its neighbours (vertices that are directly connected to it) are marked. The queue follows the First In First Out (FIFO) queuing method, and therefore, the neighbors of the node will be visited in the order in which they were inserted in the node i.e. the node that was inserted first will be visited first, and so on.

Pseudocode

```
BFS (G, s)           //Where G is the graph and s is the source node    let Q be queue.
Q.enqueue( s ) //Inserting s in queue until all its neighbour vertices are marked.

mark s as visited.   while ( Q is not empty)
    //Removing that vertex from queue,whose neighbour will be visited now    v = Q.dequeue( )

    //processing all the neighbours of v    for all neighbours w of v
in Graph G           if w is not visited
    Q.enqueue( w )    //Stores w in Q to further visit its neighbour        mark w as
visited.
```

Uniform Cost Search

The UCS algorithm uses the path's cost from the initial node to the current node as the extension criterion. Starting from the initial state (starting node), the UCS algorithm, in each step chooses the node that is closer to the initial node. When the algorithm finds the solution, it returns the path from the initial state to the final state. The UCS algorithm is characterized as complete, as it always returns a solution if it exists. Moreover, the UCS algorithm guarantees the optimum solution.

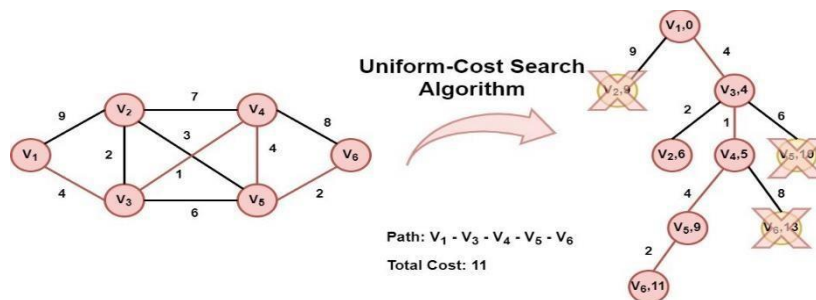


Figure: UCS

Uniform Cost Search is the best algorithm for a search problem, which can solve any general graph for optimal cost. Uniform Cost Search as it sounds searches in branches, which are more or less the same in cost.

Uniform Cost Search demands the use of a priority queue. UCS is an algorithm used to move around a directed weighted search space to go from a start node to one of the ending nodes with a minimum cumulative cost. This search

is an uninformed search algorithm since it operates in a brute-force manner, i.e. it does not take the state of the node or search space into consideration. It is used to find the path with the lowest cumulative cost in a weighted graph where nodes are expanded according to their cost of traversal from the root node. This is implemented using a priority queue where lower the cost higher is its priority.

Algorithm of Uniform Cost Search

Below is the algorithm to implement Uniform Cost Search in Artificial Intelligence:-

Algorithm for USC

1. Insert RootNode into the queue.
2. Repeat till queue is not empty:
3. Remove the next element with the highest priority from the queue.
4. If the node is a destination node, then print the cost and the path and exit, else insert all the children of removed elements into the queue with their cumulative cost as their priorities.

Here root Node is the starting node for the path, and a priority queue is being maintained to maintain the path with the least cost to be chosen for the next traversal. In case 2 paths have the same cost of traversal, nodes are considered alphabetically.

2) Solved Lab Activities:

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>2</i>	<i>2 Hours</i>	<i>Medium</i>	<i>CLO-5</i>

2) Solved Lab Activities (Allocated Time 2 Hours)

Activity 1:

The BFS search starts from a node, then it checks all the nodes at distance one from the beginning node, then it checks all the nodes at distance two, and so on. So as to recollect the nodes to be visited, BFS uses a queue.

The steps of the algorithm work as follow:

1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

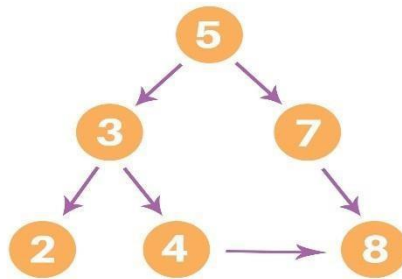


Figure: Graph for BFS Traversal Implementation

```
graph = { '5' : ['3','7'],
  '3' : ['2', '4'],
  '7' : ['8'],  '2' : [],
  '4' : ['8'],
  '8' : []
}

def bfs(visited, graph, node):
    queue = []
    visited.append(node)
    queue.append(node)
    while queue:
        m = queue.pop(0)
        print(m, end=" ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

visited = []
print("Following is the BFS Traversal ")
bfs(visited, graph, '5')
```

Output: Following is the BFS Traversal 5 3 7 2 4 8

Activity 2:

Try implementing BFS as search for a tree where start node is '1' and goal node is '7':

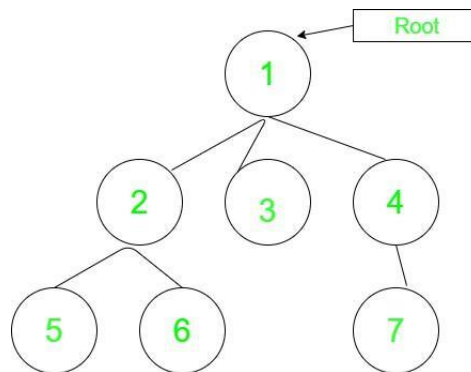


Figure: Tree for BFS Implementation

```
state_space = {
    '1' : ['2', '3', '4'],
    '2' : ['5', '6'],
    '3' : [],
    '4' : ['7'],
    '5' : [],
    '6' : [],
    '7':[]
}

def bfs(graph, start_node, end_node):

    bfs_traversal = []
    frontier = [] # a queue holding nodes that should be explored next.
    visited = [] # keeps track of nodes already visited, so we don't revisit them.

    frontier.append(start_node)
    visited.append(start_node)
    # Main BFS Loop:
    # The BFS loop runs as long as there are nodes in the frontier queue.

    while frontier:
        selected_node = frontier.pop(0) # Dequeue from frontier.
        # While there are nodes left in the frontier, remove the first element (queue
        behavior = FIFO).
    # Goal Checking
        if selected_node == end_node:
            bfs_traversal.append(selected_node)
```

```

        break

    bfs_traversal.append(selected_node)
# The current node is added to the bfs_traversal path.
# The algorithm iterates through all unvisited neighbors:
# Adds them to frontier (for future exploration).
# Marks them as visited.
    for neighbour in graph[selected_node]:
        if neighbour not in visited:
            frontier.append(neighbour)
            visited.append(neighbour)

    return bfs_traversal

start_state = "1"
goal_state = "7"
bfs_traversal = bfs(state_space, start_state, goal_state)
print("BFS Traversal:", bfs_traversal)

```

Output: BFS Traversal: ['1', '2', '3', '4', '5', '6', '7']

Activity 3 (UCS)

Imagine the same tree but this time we also mention the cost of each edge. Implement a uniform cost solution to find the path from C to B.

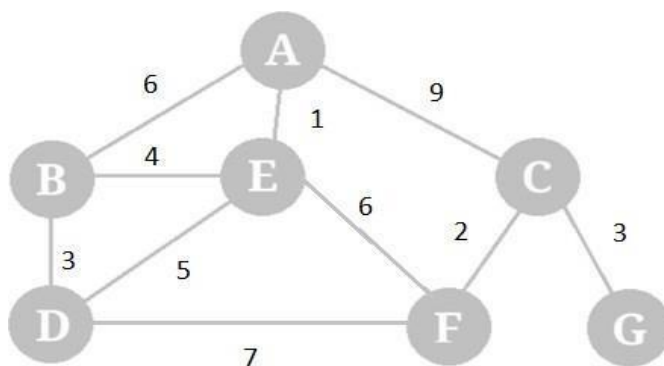


Figure - Graph

Solution:

This code implements Uniform Cost Search (UCS) — a graph search algorithm that finds the least-cost path from a start node to a goal node.

The code uses: A priority queue (heapq) to explore the lowest-cost node first.

A visited dictionary to track both the lowest known cost to each node and the parent node (for path reconstruction).

```
import heapq
# graph represented as an adjacency list
graph = {
    'A': [('B', 6), ('C', 9), ('E', 1)],
    'B': [('A', 6), ('D', 3), ('E', 4)],
    'C': [('A', 9), ('F', 2), ('G', 3)],
    'D': [('B', 3), ('E', 5), ('F', 7)],
    'E': [('A', 1), ('B', 4), ('D', 5), ('F', 6)],
    'F': [('C', 2), ('E', 6), ('D', 7)],
    'G': [('C', 3)]
}

def uniform_cost_search(graph, start, goal):
    # Priority queue to store the frontier nodes, initialized with the start node
    priority_queue = [(0, start)]
    # Dictionary to store the cost of the shortest path to each node
    visited = {start: (0, None)}

    while priority_queue:
        # Pop the node with the lowest cost from the priority queue
        current_cost, current_node = heapq.heappop(priority_queue)

        # If we reached the goal, return the total cost and the path
        if current_node == goal:
            return current_cost, reconstruct_path(visited, start, goal)

        # Explore the neighbors (Otherwise, explore all neighbors)
        for neighbor, cost in graph[current_node]:
            total_cost = current_cost + cost
            # Check if this path to the neighbor is better than any previously found
            if neighbor not in visited or total_cost < visited[neighbor][0]:
                visited[neighbor] = (total_cost, current_node)
                heapq.heappush(priority_queue, (total_cost, neighbor))

    # If the goal is not reachable, return None
    return None

def reconstruct_path(visited, start, goal):
    # Reconstruct the path from start to goal by following the visited nodes
    path = []
```

```

current = goal
while current is not None:
    path.append(current)
    current = visited[current][1] # Get the parent node
path.reverse()
return path
# Example usage of the UCS function
start_node = 'C'
goal_node = 'B'
result = uniform_cost_search(graph, start_node, goal_node)

if result:
    total_cost, path = result
    print(f"Least cost path from {start_node} to {goal_node}: {' -> '.join(path)} with total cost {total_cost}")
else:
    print(f"No path found from {start_node} to {goal_node}")

```

Output: Least cost path from C to B: C -> F -> E -> B with total cost 12

Activity 4:

Using the code in ‘activity 2’ apply the same procedure to the graph given in the Figure below start node is A and goal node is F:

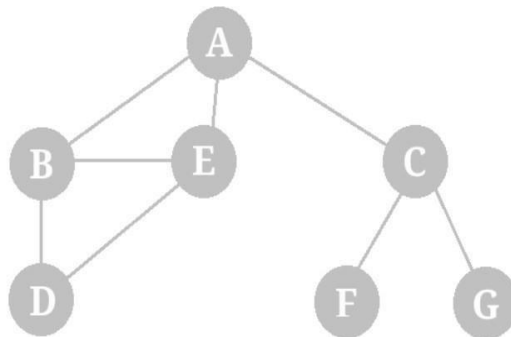


Figure: Graph for Activity 4

Activity 5:

Try changing the start and goal states for the Activity 4 implementation and observe what happens.

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

In order to move onto more complex implementations of BFS & UCS let us consider a concise version of Romania Map Problem given in Figure below:

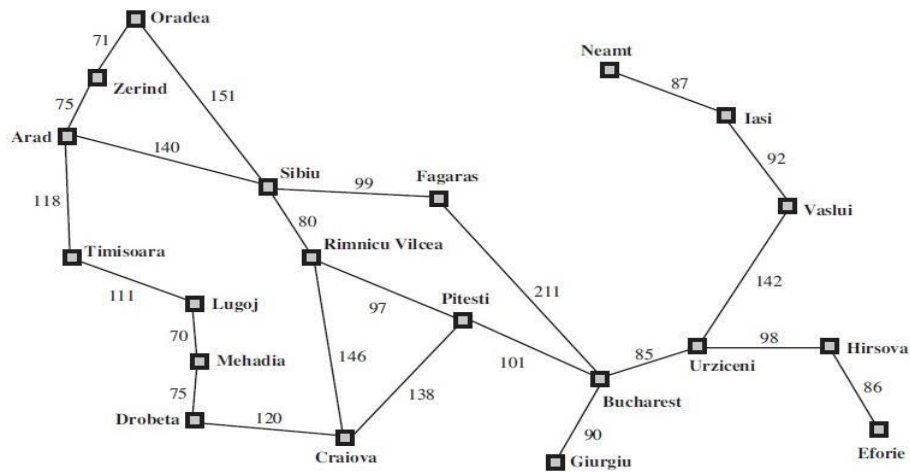


Figure: Romania Map

- Draw a corresponding graph from the above Figure from Arad to Bucharest using networkx and matplotlib library in python containing all possible nodes. (Arad as Start Node and Bucharest as Goal Node)
- Apply breadth first search to reach the goal state, your output should have the path and the total distance to reach the path.
- Solve the same Romania map given in the figure using UCS from Arad to Bucharest:

Lab Task 2:

Consider a maze as shown below. Each empty tile represents a separate node in the graph. There are a maximum of four possible actions i.e., to move up, down, left or right on any given tile/node. Using BFS, find out how to get out of the maze if you're in the start position depicted below.

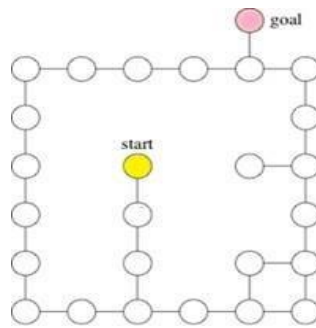
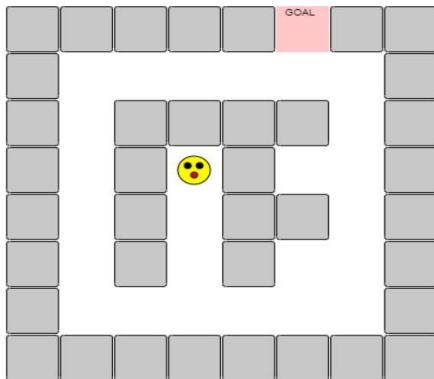


Figure - Puzzle

Lab 05 Informed Search and Greedy Best-First Search

Objective:

This lab focuses on introducing informed search, heuristics, and Greedy Best-First Search (GBFS). Unlike uninformed search algorithms such as BFS, DFS, and UCS, which explore the search space without using an estimate of how close a state is to the goal, informed search algorithms use additional problem-specific information to guide the search.

This additional information is provided through a heuristic function, $h(n)$, which estimates how close a particular state or node is to the goal. Heuristics help the search algorithm identify and prioritize promising states instead of exploring the search space without guidance.

Greedy Best-First Search is an informed search algorithm that uses the heuristic value to select the node that appears closest to the goal. At each step, it selects the frontier node with the lowest heuristic value:

$$f(n) = h(n)$$

Through this lab, students will understand how heuristic information can guide a search and implement Greedy Best-First Search in Python.

1) Useful Concepts (Allocated Time 15 Minutes)

Informed and Uninformed Search

Search algorithms can broadly be categorized as uninformed or informed search algorithms.

Uninformed search algorithms do not use additional knowledge about the location of the goal. They explore the search space using the information provided by the problem definition. Examples include:

- Depth-First Search (DFS)
- Breadth-First Search (BFS)
- Uniform Cost Search (UCS)

Informed search algorithms use additional problem-specific information to guide the search toward the goal. This information is commonly provided through a heuristic function.

Examples of informed search algorithms include:

- Greedy Best-First Search
- A* Search

What is a Heuristic?

A heuristic is an estimate used to guide a search algorithm toward a goal. A heuristic function is commonly represented as: $h(n)$ where n represents the current node or state and $h(n)$ represents the estimated cost from node n to the goal.

For example, suppose a robot needs to navigate toward a destination. The straight-line distance from its current location to the destination can be used as a heuristic.

Consider the following heuristic values:

Node	$h(n)$
A	10
B	7
C	4
D	6
G (Goal)	0

A smaller heuristic value indicates that the node is estimated to be closer to the goal. Therefore, if nodes B and C are both available for exploration, a greedy search would prefer C because:

$$h(C) = 4 < h(B) = 7$$

A heuristic is an estimate and does not necessarily represent the actual cost of reaching the goal.

Use of Heuristics

Heuristics are useful when the search space is large and exploring every possible state would be inefficient. They help the search algorithm prioritize states that appear more promising. Common examples include Route finding: straight-line distance to the destination, Grid navigation: Manhattan distance or Euclidean distance to the goal, Puzzle solving: number of misplaced tiles or estimated distance of tiles from their goal positions. Robot navigation: estimated distance between the robot's current position and target position.

Greedy Best-First Search

Greedy Best-First Search selects the frontier node having the smallest heuristic value. Its evaluation function is:

$$f(n) = h(n)$$

Therefore, GBFS asks: "**Which available node currently appears to be closest to the goal?**" It then expands that node first. For example, if the frontier contains:

Node	$h(n)$
B	8
C	3
D	5

Greedy Best-First Search will select C, since it has the smallest heuristic value.

GBFS maintains alternative nodes in its frontier so that they can be explored later if required. However, because its decision is based only on $h(n)$ and does not consider the actual path cost already incurred, it does **not guarantee the least-cost solution in general**.

2) Solved Lab Activities (Allocated Time 1.5 Hour)

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
2	2 Hours	Medium	CLO-5

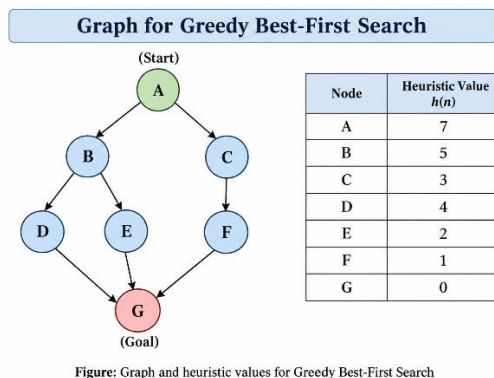
Activity Outcomes:

The expected outcomes of this lab activity are students' ability to:

- Understand informed search and the role of heuristic functions in guiding a search toward the goal.
- Understand and implement Greedy Best-First Search (GBFS) using heuristic values to select the next node for exploration.

Activity 1: Understanding the Effect of a Heuristic

Consider the following graph:



The start node is **A** and the goal node is **G**.

Initially, A is explored.

Its neighbours are: B, $h(B) = 5$ and C, $h(C) = 3$

Since C has the smaller heuristic value, GBFS selects C. After expanding C, F becomes available with $h(F) = 1$ so, F is selected next. Expanding F makes G available with $h(G) = 0$. The goal is therefore reached.

The nodes explored are:

A → C → F → G

This example demonstrates how heuristic values determine the order in which GBFS explores nodes.

```
# Graph represented using an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['G'],
    'E': ['G'],
    'F': ['G'],
    'G': []
}

# Heuristic values
heuristic = {
    'A': 7,
    'B': 5,
    'C': 3,
    'D': 4,
    'E': 2,
    'F': 1,
    'G': 0
}

def greedy_best_first(graph, heuristic, start, goal):

    # Frontier initially contains the start node
    frontier = [start]

    # Set to keep track of visited nodes
    visited = set()

    while frontier:

        # Select the node having the smallest heuristic
        current = min(frontier, key=lambda node: heuristic[node])
        frontier.remove(current)

        print(current)

        # Goal test
        if current == goal:
            print("Goal found!")
```

```
        return

    visited.add(current)

    # Add unvisited neighbours to the frontier
    for neighbour in graph[current]:
        if neighbour not in visited and neighbour not in frontier:
            frontier.append(neighbour)

# Driver Code
greedy_best_first(graph, heuristic, 'A', 'G')
```

Output

A

C

F

G

Goal found!

Observe that the algorithm does not simply follow the first available neighbour. It compares the heuristic values of the nodes available in the **frontier** and selects the node with the smallest value.

Activity 2: Effect of Changing the Heuristic Function.

Using the graph from Activity 2, change the heuristic values to:

```
heuristic = {
    'A': 7,
    'B': 2,
    'C': 5,
    'D': 1,
    'E': 3,
    'F': 2,
    'G': 0
}
```

Run the program again and observe the order in which the nodes are explored. Compare the new traversal with the previous result.

Discussion

Changing the heuristic values can change the search direction even though the graph itself remains unchanged. This demonstrates that the behavior of an informed search algorithm depends strongly on the heuristic information supplied to it.

4) Graded Lab Tasks (Allotted Time 1.5 Hour)

Task 1: Manhattan Distance as a Heuristic

For a robot moving on a Manhattan Distance as a Heuristic For a robot moving on a two-dimensional grid, use Manhattan distance as the heuristic: $h(n) = |x_1 - x_2| + |y_1 - y_2|$ Implement the following function:

```
def manhattan_distance(current, goal):  
    return abs(current[0] - goal[0]) + abs(current[1] - goal[1])
```

Use: goal = (4, 4) and positions = [(0, 0), (1, 2), (2, 3), (3, 4), (4, 4)]

Manhattan Distance as a Heuristic

(a) Grid environment with given positions

(0, 0)

(1, 2)

(2, 3)

(3, 4)

(4, 4) – Goal

Manhattan Distance

$h(n) = |x_1 - x_2| + |y_1 - y_2|$

Where:
 (x_1, y_1) = current position
 (x_2, y_2) = goal position

Example: from (1, 2) to (4, 4)

$h(n) = |4 - 1| + |4 - 2| = 3 + 2 = 5$

Manhattan Distance for Given Positions (Goal = (4, 4))

Position (x, y)	Calculation	$h(n)$
(0, 0)	$ 4 - 0 + 4 - 0 = 4 + 4$	8
(1, 2)	$ 4 - 1 + 4 - 2 = 3 + 2$	5
(2, 3)	$ 4 - 2 + 4 - 3 = 2 + 1$	3
(3, 4)	$ 4 - 3 + 4 - 4 = 1 + 0$	1
(4, 4)	$ 4 - 4 + 4 - 4 = 0 + 0$	0

A smaller Manhattan distance indicates that the position is closer to the goal.

Task 2: Solve an 8-Puzzle Using Greedy Best-First Search

The 8-puzzle is a classical AI search problem consisting of a (3 X 3) grid containing eight numbered tiles and one empty space. A tile adjacent to the empty space can be moved into it. The possible movements of the empty space are up, down, left, or right, provided the movement remains within the grid. Each arrangement of the puzzle represents a **state** in the search space. The objective is to transform a given **initial state** into the specified **goal state** by selecting a sequence of valid moves.

In this task, use Greedy Best-First Search (GBFS) to solve the puzzle. GBFS selects the next state for exploration according to its heuristic value ($h(n)$). The state with the smallest heuristic value is considered more promising and is selected first.

Heuristic Function

$h(n)$ = number of misplaced tiles

A tile is considered misplaced if it is not in the position specified by the goal state. Do not include the blank tile when calculating the heuristic.

For example, consider the state:

```

1   2   3
4   _   6
7   5   8

```

For the given goal state, tiles **5 and 8** are misplaced. Therefore: $h(n)=2$

Expected output:

- The sequence of states from start to goal
- The heuristic value of each explored state
- The total number of steps in the solution.

Initial State			Goal State		
1	2	3	1	2	3
4		6	4	5	6
7	5	8	7	8	

Task 3: Solve an 8-Puzzle given by user input Using Greedy Best-First Search

You can improve task 2 by allowing user to input any start state and then the program would first calculate whether the puzzle is solvable or not (explore for that) then it performs the rest of the calculations and search.

Lab 06 Informed Searching Algorithms: A* Search

Objective:

This lab will introduce students to heuristic search methods. In particular we will implement a particular variant of heuristic search known as A* search. Students will also be introduced to different approaches to optimize heuristically inclined search problems.

1) Useful Concepts (Allocated Time 30Minutes)

The A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals.

The A* Search algorithms, unlike other traversal techniques uses intelligence. And it is also worth mentioning that many games and web-based maps use this algorithm to find the shortest path very efficiently.

Consider a square grid having many obstacles and we are given a starting cell and a target cell. We want to reach the target cell (if possible) from the starting cell as quickly as possible.

What A* Search Algorithm does is that at each step it picks the node according to a value-‘f’ which is a parameter equal to the sum of two other parameters – ‘g’ and ‘h’. At each step it picks the node/cell having the lowest ‘f’, and process that node/cell. We define ‘g’ and ‘h’ as below:

g = the movement cost to move from the starting point to a given square on the grid, following the path generated to get there.

h = the estimated movement cost to move from that given square on the grid to the final destination. This is often referred to as the heuristic, which is nothing but a kind of smart guess. We really don’t know the actual distance until we find the path, because all sorts of things can be in the way (walls, water, etc.). There can be many ways to calculate this ‘h’ which are discussed in the later sections.

We can understand the working of A* search by the Algorithm shown in Figure below:

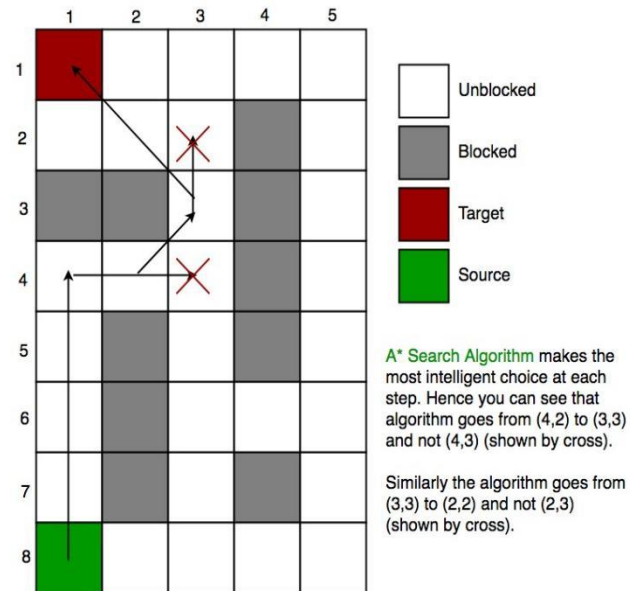


Figure: Graph for path finding.

To understand the implementation we create two lists:

A* Search Algorithm

A* Search uses two lists:

- **OPEN list:** contains nodes that have been generated but not yet expanded.
- **CLOSED list:** contains nodes that have already been expanded.

For each node n :

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = actual path cost from the start node to n
- $h(n)$ = estimated cost from n to the goal
- $f(n)$ = estimated total cost of a solution path through n

Algorithm

A* uses two lists:

- **OPEN list:** contains nodes that have been generated but not yet expanded.

-
-
- **CLOSED list:** contains nodes that have already been expanded.

For each node n :

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual path cost from the start node to n , and $h(n)$ is the estimated cost from n to the goal.

Algorithm

1. Initialize the search.

Create empty **OPEN** and **CLOSED** lists. For the start node, set:

$$\begin{aligned}g(start) &= 0 \\h(start) &= heuristic(s, startgoal) \\f(start) &= g(start) + h(start)\end{aligned}$$

Add the start node to the **OPEN list**.

2. Select the best node.

From the OPEN list, select the node having the smallest $f(n)$. Call this node **q** and remove it from OPEN.

3. Check for the goal.

If **q** is the goal node, stop the search and reconstruct the solution path using the parent links.

4. Generate successors.

Generate all valid successors (neighbours) of **q**.

5. Evaluate each successor.

For each successor, calculate:

$$\begin{aligned}g(successor) &= g(q) + cost(q, successor) \\h(successor) &= heuristic(s, successorgoal) \\f(successor) &= g(successor) + h(successor)\end{aligned}$$

Then:

- If the same state exists in **OPEN** with an equal or lower g -cost, ignore the new successor.
- If the same state exists in **CLOSED** with an equal or lower g -cost, ignore the new successor.
- Otherwise, add the successor to **OPEN**. If a better path to an existing state has been found, update its cost and set its parent to **q**.

6. Close the current node.

Add **q** to the **CLOSED** list.

7. Repeat the search.

If the OPEN list is not empty, return to **Step 2**. If OPEN becomes empty before the goal is reached, report that no solution was found.

Heuristic Functions

The heuristic $h(n)$ estimates the remaining cost from the current state to the goal. Depending on the problem, different heuristic functions may be used.

For grid-based pathfinding, common distance heuristics include:

- **Manhattan Distance**
- **Euclidean Distance**
- **Diagonal Distance**

The suitability of a heuristic depends on the allowed movements and the problem being solved.

2) Solved Lab Activities (Allotted Time 1.5 Hour)

Lets Understand different types of heuristically inclined methods and calculate Heuristics

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>2</i>	<i>2 Hours</i>	<i>Medium</i>	<i>CLO-5</i>

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Implement A* Search

Activity 1: Understanding Heuristics for A* Search

A* is an **informed search algorithm** that evaluates each node using:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual path cost from the start node to the current node, and $h(n)$ is an estimate of the remaining cost from the current node to the goal.

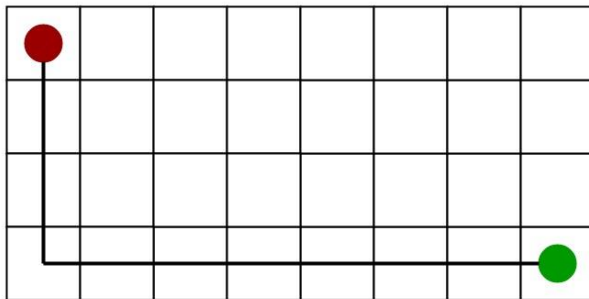
The heuristic function should reflect the type of movement allowed in the search space. Common distance-based heuristics include the following.

1. Manhattan Distance

Manhattan distance is suitable when movement is allowed only **up, down, left, and right**.

$$h(n) = |x_g - x_n| + |y_g - y_n|$$

The Manhattan Distance Heuristics is shown by the below figure (assume red spot as source cell and green spot as target cell).



```
def manhattan_distance(current, goal):  
    return abs(current[0] - goal[0]) +  
    abs(current[1] - goal[1])  
  
current = (1, 1)  
goal = (9, 9)  
  
print(manhattan_distance(current, goal))
```

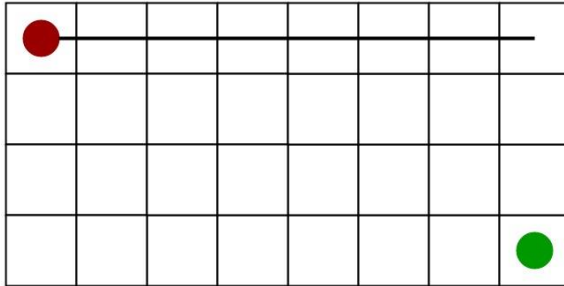
Output:

10

2. Diagonal Distance

When diagonal movement is also allowed (**movement in 8 directions possible**) and each move has equal unit cost, diagonal distance can be calculated as:

$$h(n) = \max(|x_g - x_n|, |y_g - y_n|)$$



```
def diagonal_distance(current, goal):
    dx = abs(current[0] - goal[0])
    dy = abs(current[1] - goal[1])
    return max(dx, dy)

current = (1, 1)
goal = (4, 8)

print(diagonal_distance(current, goal))
```

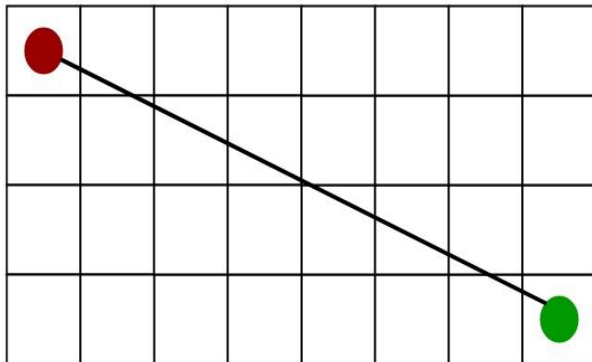
Output:

7

3. Euclidean Distance

Euclidean distance represents the straight-line distance between two points and is useful when movement is not restricted to horizontal and vertical grid directions.

$$h(n) = \sqrt{(x_g - x_n)^2 + (y_g - y_n)^2}$$



```
import math
def euclidean_distance(current, goal):
    return math.sqrt(
        (goal[0] - current[0]) ** 2 +
        (goal[1] - current[1]) ** 2
    )

current = (1, 1)
goal = (4, 8)

print(euclidean_distance(current, goal))
```

Output:

7.615773105863909

The choice of heuristic depends on the movement rules of the problem. A suitable heuristic helps A* estimate which nodes are more promising while searching for a path to the goal.

Activity 2: Maze Path Planning Using A* Search

Consider the maze shown below. Each **white or green cell** represents a **traversable node**, while the **blue cells** represent **obstacles** that cannot be crossed. The starting node is **A** and the goal node is **Y**. Movement is allowed only **up, down, left, and right**, and each move has a **unit cost of 1**.

Use **A* Search** to find a path from A to Y. Since movement is restricted to four directions, use **Manhattan Distance** as the heuristic:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

A* evaluates each node using:

$$f(n) = g(n) + h(n)$$

5			W		X	Y
4	R	S	T	U		V
3	M	N		O	P	Q
2	H	I	J		K	L
1	F		G			
0	A		B	C	D	E
	0	1	2	3	4	5

where $g(n)$ is the actual path cost from A to the current node and $h(n)$ is the estimated remaining distance to Y.

For the given maze, one optimal path obtained is:

$A \rightarrow F \rightarrow H \rightarrow M \rightarrow R \rightarrow S \rightarrow T \rightarrow U \rightarrow O \rightarrow P \rightarrow Q \rightarrow V \rightarrow Y$

The total path cost is:

$$g(Y) = 12$$

because the path contains **12 moves**.

```
import heapq

# Coordinates of traversable nodes
nodes = {
    'A': (0, 0),
    'B': (2, 0),
    'C': (3, 0),
    'D': (4, 0),
    'E': (5, 0),

    'F': (0, 1),
    'G': (2, 1),

    'H': (0, 2),
```

```

'I': (1, 2),
'J': (2, 2),
'K': (4, 2),
'L': (5, 2),

'M': (0, 3),
'N': (1, 3),
'O': (3, 3),
'P': (4, 3),
'Q': (5, 3),

'R': (0, 4),
'S': (1, 4),
'T': (2, 4),
'U': (3, 4),
'V': (5, 4),

'W': (2, 5),
'X': (4, 5),
'Y': (5, 5)
}

# Graph connections based on valid up/down/left/right moves
graph = {
    'A': ['F'],
    'B': ['C', 'G'],
    'C': ['B', 'D'],
    'D': ['C', 'E'],
    'E': ['D'],

    'F': ['A', 'H'],
    'G': ['B', 'J'],

    'H': ['F', 'I', 'M'],
    'I': ['H', 'J', 'N'],
    'J': ['I', 'G'],
    'K': ['L', 'P'],
    'L': ['K', 'Q'],

    'M': ['H', 'N', 'R'],
    'N': ['M', 'I', 'S'],
    'O': ['P', 'U'],
    'P': ['O', 'Q', 'K'],
    'Q': ['P', 'L', 'V'],

    'R': ['M', 'S'],

```

```

'S': ['R', 'T', 'N'],
'T': ['S', 'U', 'W'],
'U': ['T', 'O'],
'V': ['Q', 'Y'],

'W': ['T'],
'X': [],
'Y': ['V']
}

def manhattan_distance(node, goal):
    x1, y1 = nodes[node]
    x2, y2 = nodes[goal]
    return abs(x1 - x2) + abs(y1 - y2)

def a_star(graph, start, goal):
    open_list = []

    # (f-value, node)
    heapq.heappush(
        open_list,
        (manhattan_distance(start, goal), start)
    )

    g_cost = {start: 0}
    parent = {start: None}
    closed = set()

    while open_list:

        current_f, current = heapq.heappop(open_list)

        if current in closed:
            continue

        print(
            "Node:", current,
            "g =", g_cost[current],
            "h =", manhattan_distance(current, goal),
            "f =", current_f
        )

        # Goal test
        if current == goal:
            path = []

```

```

        while current is not None:
            path.append(current)
            current = parent[current]

        path.reverse()
        return path

    closed.add(current)

    # Explore neighbours
    for neighbour in graph[current]:

        new_g = g_cost[current] + 1

        if neighbour not in g_cost or new_g < g_cost[neighbour]:

            g_cost[neighbour] = new_g

            h = manhattan_distance(neighbour, goal)
            f = new_g + h

            parent[neighbour] = current

            heapq.heappush(open_list, (f, neighbour))

    return None

# Driver code
start = 'A'
goal = 'Y'

path = a_star(graph, start, goal)

print("\nFinal Path:")
print(path)

if path:
    print("Total Path Cost:", len(path) - 1)
else:
    print("No path found.")

```

Output:

Node: A g = 0 h = 10 f = 10

Node: F g = 1 h = 9 f = 10

Node: H $g = 2$ $h = 8$ $f = 10$
Node: I $g = 3$ $h = 7$ $f = 10$
Node: J $g = 4$ $h = 6$ $f = 10$
Node: M $g = 3$ $h = 7$ $f = 10$
Node: N $g = 4$ $h = 6$ $f = 10$
Node: R $g = 4$ $h = 6$ $f = 10$
Node: S $g = 5$ $h = 5$ $f = 10$
Node: T $g = 6$ $h = 4$ $f = 10$
Node: U $g = 7$ $h = 3$ $f = 10$
Node: W $g = 7$ $h = 3$ $f = 10$
Node: G $g = 5$ $h = 7$ $f = 12$
Node: O $g = 8$ $h = 4$ $f = 12$
Node: P $g = 9$ $h = 3$ $f = 12$
Node: Q $g = 10$ $h = 2$ $f = 12$
Node: V $g = 11$ $h = 1$ $f = 12$
Node: Y $g = 12$ $h = 0$ $f = 12$

Final Path:

['A', 'F', 'H', 'I', 'N', 'S', 'T', 'U', 'O', 'P', 'Q', 'V', 'Y']

Total Path Cost: 12

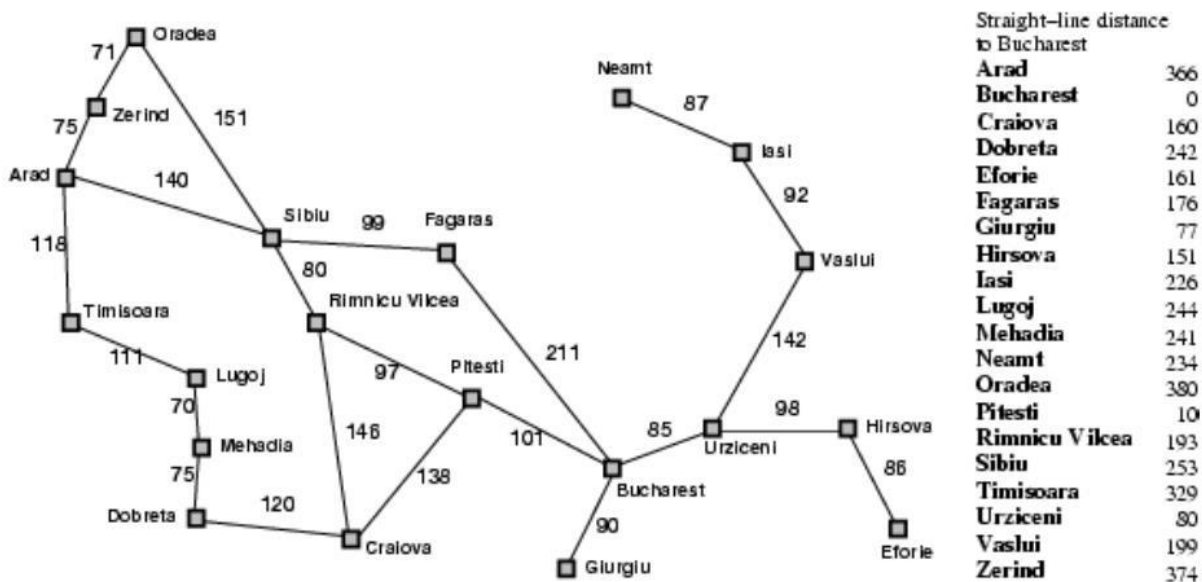
Note: A may return a different path when multiple paths have the same minimum cost. The selected path can depend on how ties between nodes having equal $f(n)$ values are resolved.*

3) Graded Lab Tasks (Allotted Time 1 Hour)

Lab Task 1:

Implement the A* search algorithm to find the path from Arad to Bucharest, show what improvements are evident when using Heuristics based search as compared to uninformed methods. What improvements did you notice?

Heuristic for the Romania problem

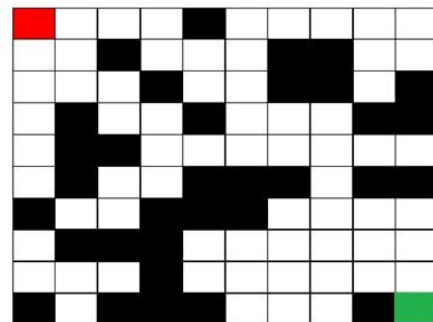


Lab Task 2:

Modify the 8-puzzle problem from the previous GBFS lab task to implement A* Search. Consider each tile move to have a unit path cost (cost = 1).

Lab Task 3:

Your goal is to navigate a robot out of a maze. The robot starts in the corner of the maze marked with red color. You can turn the robot to face north, east, south, or west. You can direct the robot to move forward a certain distance, although it will stop before hitting a wall. The goal is to reach the final state marked with green color. Write a program that implements A* algorithms to solve this maze. Write the path followed (in the form of coordinates) and the cost of the path.



Lab 07 Local Search Algorithms: Hill Climbing , Simulated Annealing

Traditional search algorithms such as BFS, DFS, UCS, and A* systematically explore the search space and maintain information about multiple states. In many optimization problems, however, the path to the solution is less important than finding a good solution itself. Local search addresses such problems by starting from a candidate state and improving it through neighboring states without maintaining a complete search tree.

Hill Climbing is a local search algorithm that starts from an initial state and repeatedly moves to a neighboring state with a better evaluation value. Unlike Best-First Search, it does not maintain a frontier of alternative states; instead, it makes decisions based on the neighbors of the current state. The process continues until the goal is reached or no neighboring state provides an improvement.

Hill Climbing is a simple local search algorithm that repeatedly moves from the current state to a better neighboring state until no further improvement is possible. Common variants include Simple Hill Climbing, which selects the first improving neighbor; Steepest-Ascent Hill Climbing, which selects the best available neighbor; and Stochastic Hill Climbing, which introduces randomness in selecting an improving move.

Since Hill Climbing relies on local information, it may become trapped at a local optimum, plateau, or ridge instead of reaching the global optimum. Simulated Annealing helps overcome this limitation by occasionally accepting worse states. At high temperature, such moves are more likely, helping the search escape local optima; as the temperature decreases, the algorithm increasingly favors better states and moves toward convergence.

1) Useful Concepts (Allocated Time 15Minutes)

Hill Climbing

Hill Climbing is a heuristic search used for mathematical optimization problems in the field of Artificial Intelligence. Given a large set of inputs and a good heuristic function, it tries to find a sufficiently good solution to the problem. This solution may not be the global optimal maximum. It Uses the Greedy approach: At any point in state space, the search moves in that direction only which optimizes the cost of function with the hope of finding the optimal solution at the end.

There are different types of Hill Climbing approaches that can be used

-
-
1. **Simple Hill climbing:** It examines the neighboring nodes one by one and selects the first neighboring node which optimizes the current cost as the next node.

Algorithm:

Step 1 : Evaluate the initial state. If it is a goal state then stop and return success. Otherwise, make initial state as current state.

Step 2 : Loop until the solution state is found or there are no new operators present which can be applied to the current state.

- a) Select a state that has not been yet applied to the current state and apply it to produce a new state.
- b) Perform these to evaluate new state

i. If the current state is a goal state, then stop and return success. ii. If it is better than the current state, then make it current state and proceed further. iii. If it is not better than the current state, then continue in the loop until a solution is found. **Step 3 :** Exit.

2. **Steepest-Ascent Hill climbing:** It first examines all the neighboring nodes and then selects the node closest to the solution state as of the next node.

Algorithm:

Step 1 : Evaluate the initial state. If it is a goal state then stop and return success. Otherwise, make initial state as current state.

Step 2 : Repeat these steps until a solution is found or current state does not change

- a) Select a state that has not been yet applied to the current state.
- b) Initialize a new 'best state' equal to current state and apply it to produce a new state.
- c) Perform these to evaluate new state

i. If the current state is a goal state, then stop and return success. ii. If it is better than best state, then make it best state else continue loop with another new state. d) Make best state as current state and go to Step 2: b) part. **Step 3 :** Exit

3. **Stochastic hill climbing:** It does not examine all the neighboring nodes before deciding which node to select. It just selects a neighboring node at random and decides (based on the amount of improvement in that neighbor) whether to move to that neighbor or to examine another.

Algorithm:

Step 1: Evaluate the initial state. If it is a goal state then stop and return success. Otherwise, make the initial state the current state.

Step 2: Repeat these steps until a solution is found or the current state does not change.

a) Select a state that has not been yet applied to the current state.

b) Apply successor function to the current state and generate all the neighbor states.

c) Among the generated neighbor states which are better than the current state choose a state randomly (or based on some probability function).

d) If the chosen state is the goal state, then return success, else make it the current state and repeat step 2: b) part.

Step 3: Exit.

2) Solved Lab Activities (Allotted Time 1.5 Hour)

Sr.No	Allocated Time	Level of Complexity	CLO Mapping
2	2 Hours	Medium	CLO-5

Activity Outcomes: Implementing Hill Climbing and simulated annealing

Activity 1

Implementation of hill climbing algorithm in python is given below. In this implementation the algorithm is searching for the most efficient tour to visit a number of cities.

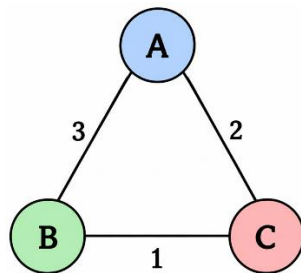


Figure: Distance between cities for Hill Climbing search

Possible Tours:

A --[3 km]--> B --[1 km]--> C | [4 km]

A --[2 km]--> C --[1 km]--> B | [3 km] (most efficient) .

Distances between cities

```
distance = {
    'A': {'B': 3, 'C': 2},
    'B': {'A': 3, 'C': 1},
    'C': {'A': 2, 'B': 1}
}

# Calculate the total distance of a tour
def tour_distance(tour):
    total = 0

    for i in range(len(tour) - 1):
        total += distance[tour[i]][tour[i + 1]]

    return total

def hill_climbing():

    # Initial solution
    current = ['A', 'B', 'C']
    current_cost = tour_distance(current)

    print("Initial tour:", current)
    print("Distance:", current_cost, "km")

    # Generate a neighbouring solution
    neighbour = ['A', 'C', 'B']
    neighbour_cost = tour_distance(neighbour)

    print("\nNeighbour tour:", neighbour)
    print("Distance:", neighbour_cost, "km")

    # Move to the better solution
    if neighbour_cost < current_cost:
        current = neighbour
        current_cost = neighbour_cost

    return current, current_cost

best_tour, best_distance = hill_climbing()

print("\nBest tour:", best_tour)
print("Best distance:", best_distance, "km")
```

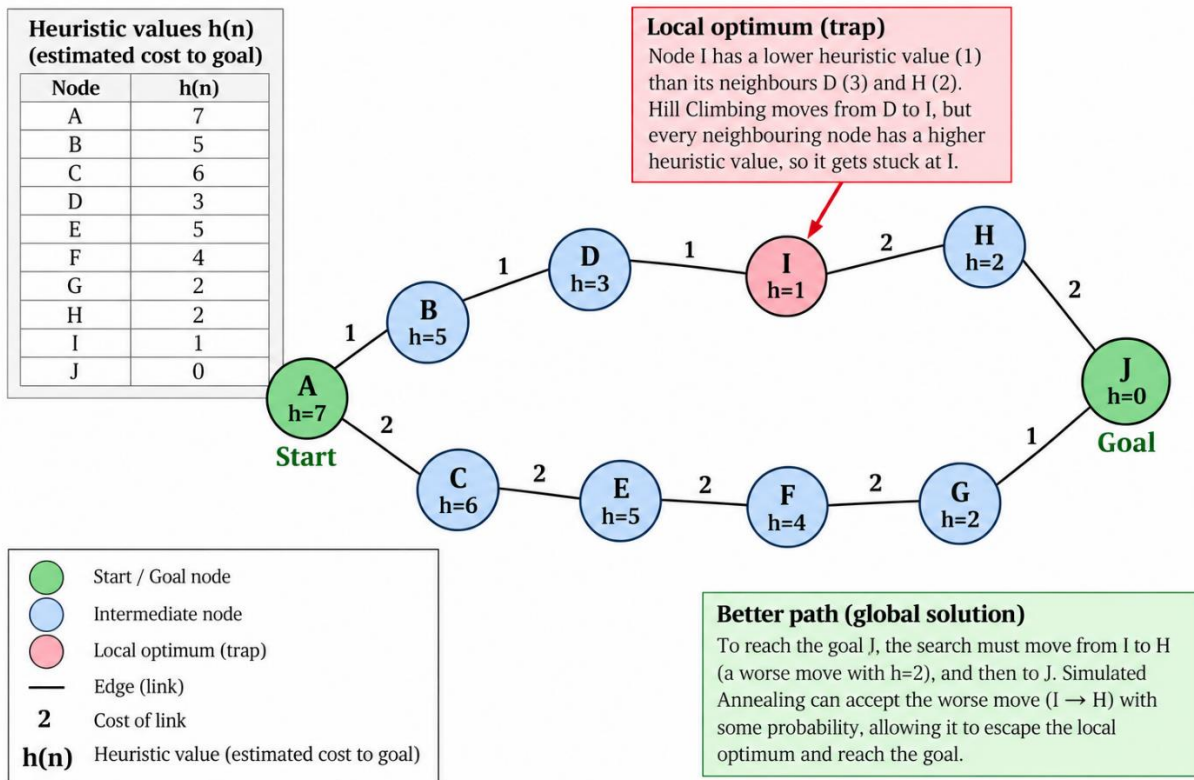
Output:

Initial tour: ['A', 'B', 'C']
Distance: 4 km
Neighbour tour: ['A', 'C', 'B']
Distance: 3 km
Best tour: ['A', 'C', 'B']
Best distance: 3 km

We can try experimenting with different thresholds to see where the optimum solution is achieved.

Activity 2:

Consider the graph shown in the figure. Each edge represents a possible move and its associated cost, while $h(n)$ represents the estimated distance of a node from the goal. The objective is to move from A to J, where a smaller heuristic value indicates a state closer to the goal.



Let's first apply Hill Climbing Search, selecting at each step the neighboring node having the lowest heuristic value. Observe whether the algorithm reaches the goal or becomes trapped at a local optimum.

Use this graph representation:

```
graph = {
    'A': {'B': 1, 'C': 2},
    'B': {'A': 1, 'D': 1},
    'C': {'A': 2, 'E': 2},
    'D': {'B': 1, 'I': 1},
    'E': {'C': 2, 'F': 2},
    'F': {'E': 2, 'G': 2},
    'G': {'F': 2, 'J': 1},
    'H': {'I': 2, 'J': 2},
    'I': {'D': 1, 'H': 2},
    'J': {'G': 1, 'H': 2}
}

heuristic = {
    'A': 7,
    'B': 5,
    'C': 6,
    'D': 3,
    'E': 5,
    'F': 4,
    'G': 2,
    'H': 2,
    'I': 1,
    'J': 0
}
```

Step 1: Hill Climbing

The algorithm examines only the neighbors of the current node and moves to the neighbor having a lower heuristic value.

```
def hill_climbing(graph, heuristic, start, goal):

    current = start
    path = [current]

    while current != goal:

        neighbours = list(graph[current].keys())

        # Select neighbour with smallest heuristic
        best = min(neighbours, key=lambda node: heuristic[node])

        print("Current:", current,
```

```

        "h =", heuristic[current],
        "Best neighbour:", best,
        "h =", heuristic[best])

    # Stop if no improvement is possible
    if heuristic[best] >= heuristic[current]:
        print("No better neighbour found.")
        print("Hill Climbing is stuck at:", current)
        return path

    current = best
    path.append(current)

return path

path = hill_climbing(graph, heuristic, 'A', 'J')
print("\nHill Climbing Path:", path)

```

The search proceeds as:

$$A(7) \rightarrow B(5) \rightarrow D(3) \rightarrow I(1)$$

At I, the available neighbors are:

$$D: h = 3, H: h = 2$$

Both are worse than $h(I) = 1$. Therefore, Hill Climbing refuses to move and becomes trapped at I, even though the goal is reachable through:

$$I \rightarrow H \rightarrow J$$

This demonstrates a local optimum.

Step 2: Simulated Annealing

Simulated Annealing modifies this behavior by sometimes accepting a worse move. For a heuristic that should be minimized, the change can be calculated as:

$$\Delta h = h(\text{next}) - h(\text{current})$$

A better move has $\Delta h < 0$ and is always accepted. A worse move may be accepted with probability:

$$P = e^{-\Delta h/T}$$

where T is the temperature.

```
graph = {
    'A': {'B': 1, 'C': 2},
    'B': {'A': 1, 'D': 1},
    'C': {'A': 2, 'E': 2},
    'D': {'B': 1, 'I': 1},
    'E': {'C': 2, 'F': 2},
    'F': {'E': 2, 'G': 2},
    'G': {'F': 2, 'J': 1},
    'H': {'I': 2, 'J': 2},
    'I': {'D': 1, 'H': 2},
    'J': {'G': 1, 'H': 2}
}

heuristic = {
    'A': 7,
    'B': 5,
    'C': 6,
    'D': 3,
    'E': 5,
    'F': 4,
    'G': 2,
    'H': 2,
    'I': 1,
    'J': 0
}

import math
import random

def simulated_annealing(graph, heuristic, start, goal):

    random.seed(2)

    current = start
    path = [current]

    temperature = 10
    cooling_rate = 0.90
```

```

while current != goal and temperature > 0.1:

    neighbours = list(graph[current].keys())

    # Choose a random neighbour
    next_node = random.choice(neighbours)

    difference = heuristic[next_node] - heuristic[current]

    # Accept a better move
    if difference < 0:
        accept = True

    # Sometimes accept a worse move
    else:
        probability = math.exp(-difference / temperature)
        accept = random.random() < probability

    if accept:
        current = next_node
        path.append(current)

    temperature = temperature * cooling_rate

    print("Current:", current,
          "h =", heuristic[current],
          "Temperature =", round(temperature, 2))

return path

path = simulated_annealing(
    graph, heuristic, 'A', 'J'
)

def path_cost(graph, path):
    cost = 0

    for i in range(len(path) - 1):
        cost += graph[path[i]][path[i + 1]]

    return cost

print("\nSimulated Annealing Path:", path)
print("Path Cost:", path_cost(graph, path))

```

The important difference is what happens at I. Hill Climbing rejects:

$$I(1) \rightarrow H(2)$$

because the heuristic becomes worse. Simulated Annealing can accept this temporary worsening move. Once at H:

$$H(2) \rightarrow J(0)$$

is an improvement, allowing the goal to be reached.

Hill Climbing stops at I because every immediate move appears worse, whereas Simulated Annealing can temporarily accept the worse move from I to H and thereby escape the local optimum and reach J.

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

Apply Hill Climbing to find the path from Arad to Bucharest, “observe how different heuristic values or starting conditions affect the solution obtained

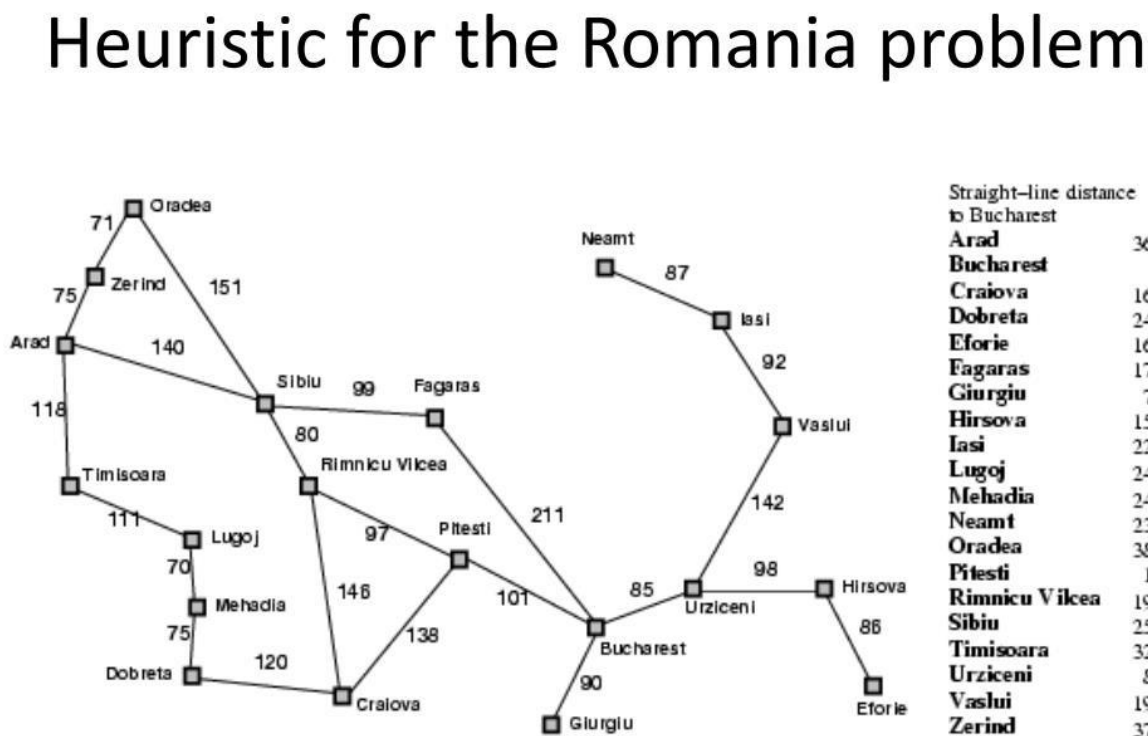


Figure: Romania road map

Lab Task 2:

Apply Hill Climbing Search to the maze shown in the figure below. Each traversable tile represents a node, while the blue tiles represent walls that cannot be crossed. The starting node is **A** and the goal node is **Y**. At each step, evaluate the neighboring nodes using an appropriate heuristic and move to the neighbor that appears closest to the goal. Continue the search until the goal is reached or no neighboring node provides an improvement.

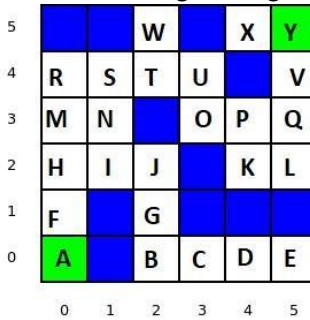


Figure: Maze Problem

Lab 08 Midterm